

Freie Universität Berlin

Bachelorarbeit am Institut für Informatik der Freien Universität Berlin

Arbeitsgruppe Cybersecurity and AI

Reduzierung von False Positives in der Cloud-Sicherheit durch Einbeziehung von Log-Kontext in eine metrikbasierte Detection-Pipeline: Eine empirische Untersuchung auf CloudAnoBench

Nirosh Heintze

Matrikelnummer: 5585800

niroh04@zedat.fu-berlin.de

Betreuer: Hamed H. Fard, M.Sc.

Eingereicht bei: Prof. Dr.-Ing. habil. Gerhard Wunder

Zweitgutachter: Prof. Dr. Marian Margraf

Berlin, 16. Juli 2026

Kurzfassung

Automatisierte Anomalie-Erkennung in Cloud-Infrastrukturen erzeugt häufig eine hohe Rate an Fehlalarmen (*False Positives*), die Analysten belasten und die Reaktionsfähigkeit auf echte Sicherheitsvorfälle beeinträchtigen. Diese Arbeit untersucht auf dem CloudAnoBench-Datensatz, inwiefern Log-Kontext die False-Positive-Rate einer metrikbasierten Detection-Pipeline reduzieren kann. Dazu wird eine reproduzierbare Case-Level-Pipeline entwickelt, die einen XGBoost-Klassifikator auf aggregierten Systemmetriken, eine log-basierte TF-IDF-Repräsentation mit logistischer Regression sowie zwei einfache Fusionsvarianten vergleicht. Die finale Evaluation erfolgt leakage-sicher mit einem scenario-aware Train-/Validierungs-/Test-Split. Early Stopping und Threshold-Wahl werden ausschließlich auf dem Validierungsset vorgenommen, während das Testset nur der finalen Auswertung dient.

Die Ergebnisse zeigen, dass die metrikbasierte Baseline auf Case-Ebene zwar alle Angriffs-Cases erkennt ($\text{Recall}_{\text{mali}} = 1,0000$), aber zugleich alle negativen Test-Cases als positiv klassifiziert ($\text{FPR}_{\text{total}} = 1,0000$). Die log-basierte Komponente liefert die beste Trennleistung und reduziert die False-Positive-Rate auf 0,2537 bei einem Test-Recall von 0,9375. Getrennt nach Negativtypen ergeben sich $\text{FPR}_{\text{anom}} = 0,2368$ und $\text{FPR}_{\text{norm}} = 0,2636$. Die AND-Fusion ist funktional identisch zur Log-only-Variante, da der metrische Trigger bei allen 253 Test-Cases aktiv ist. Die Soft-Fusion erzielt denselben Recall wie Log-only, verschlechtert sich jedoch in Precision, F1, FPR und AP. Damit trägt auf diesem Benchmark primär der Log-Kontext zur False-Positive-Reduktion bei, während die untersuchten Metrik-Aggregate und Fusionsregeln keinen zusätzlichen Nutzen liefern. Die angestrebte Recall-Schwelle von 0,95 wird im finalen Testsplit knapp verfehlt. Dieser Befund wird als zentrale Limitation diskutiert.

Eidesstattliche Erklärung

Ich versichere hiermit an Eides Statt, dass diese Arbeit von niemand anderem als meiner Person verfasst worden ist. Alle verwendeten Hilfsmittel wie Berichte, Bücher, Internetseiten oder ähnliches sind im Literaturverzeichnis angegeben, Zitate aus fremden Arbeiten sind als solche kenntlich gemacht. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungskommission vorgelegt und auch nicht veröffentlicht.

16. Juli 2026

Nirosh Heintze

Inhaltsverzeichnis

Abbildungsverzeichnis	6
Tabellenverzeichnis	6
Abkürzungsverzeichnis	7
1 Einleitung	8
1.1 Motivation	8
1.2 Problemstellung	8
1.3 Zielsetzung und Forschungsfrage	8
1.4 Beiträge der Arbeit	9
1.5 Aufbau der Arbeit	10
2 Grundlagen und verwandte Arbeiten	10
2.1 Cloud-Anomalieerkennung und Alert Fatigue	10
2.2 Metrikbasierte und logbasierte Anomalieerkennung	11
2.3 Unbalancierte Klassifikation und Precision-Recall-Metriken	12
2.4 XGBoost für tabellarische Metrikdaten	12
2.5 TF-IDF und logistische Regression für Logdaten	13
2.6 Data Leakage und gruppenbasierte Validierung	13
2.7 Klassifikatorfusion und Case-Level-Evaluation	14
3 Datensatz und Datenaufbereitung	14
3.1 CloudAnoBench und Datenbasis	14
3.2 Datenqualität und Preprocessing	15
3.3 Kanonisches Feature-Set und universelle Features	16
3.4 Leakage-Vermeidung und Split-Strategie	17
4 Metrikbasierte Baseline	18
4.1 Modellwahl und Hyperparameter	18
4.2 Schema-Leakage, Feature-Reduktion und Case-Level-Aggregation	19
4.3 Ergebnisse der Metrik-Baseline	19
5 Log-basierte Verifikation	21
5.1 Log-Daten auf Case-Ebene	21
5.2 Log-Normalisierung	21
5.3 TF-IDF-Vektorisierung	22
5.4 Logistische Regression	22
5.5 Ergebnisse der Log-Komponente	24
5.6 Threshold-Sensitivity-Analyse	25
6 Hybride Fusion und Evaluation	25
6.1 Fusion-Eingaben auf Case-Ebene	25
6.2 AND-Gate-Fusion und Soft-Score	26
6.3 Ablationsstudie	26

6.4	Fehleranalyse auf Case-Ebene	28
6.4.1	False-Negative-Cases	28
6.4.2	False-Positive-Cases nach Szenario	29
6.5	AND-Gate-Degeneration	30
6.6	Qualitätssicherung und Reproduzierbarkeit	31
7	Diskussion und Limitationen	32
7.1	Beantwortung der Forschungsfrage	32
7.2	Rolle des Log-Kontexts	33
7.3	Grenzen der Metrik-Komponente	33
7.4	Methodische Limitationen	33
7.5	Skalierbarkeit als konzeptuelle Einordnung	35
8	Fazit und Ausblick	36
8.1	Zusammenfassung	36
8.2	Zentrale Ergebnisse	36
8.3	Beantwortung der Forschungsfrage	37
8.4	Methodische Bedeutung	37
8.5	Limitationen	38
8.6	Ausblick	39
A	Anhang	41
A.1	Dokumentation der KI-Nutzung	42

Abbildungsverzeichnis

- Abbildung 1: Verteilung der fünf universellen Features nach Datensatztyp
Abbildung 2: Precision-Recall-Kurve der Metrik-Baseline
Abbildung 3: Feature-Importances des XGBoost-Modells
Abbildung 4: Top-20 TF-IDF-Terme nach Koeffizientengröße
Abbildung 5: Precision-Recall-Kurve der Log-Komponente
Abbildung 6: PR-Kurven der Case-Level-Ablation
Abbildung 7: Vergleich von FPR_{anom} und FPR_{norm} zwischen den Varianten
Abbildung 8: False Positives nach Szenario für Log-only

Tabellenverzeichnis

- Tabelle 1: Datenbasis: Zeilen und Cases nach Typ
Tabelle 2: Gruppenbasierter Train/Validation/Test-Split
Tabelle 3: XGBoost-Hyperparameter
Tabelle 4: Ergebnisse der Metrik-Baseline
Tabelle 5: Regelbasierte Log-Normalisierung
Tabelle 6: TF-IDF-Konfiguration
Tabelle 7: Hyperparameter der logistischen Regression
Tabelle 8: Ergebnisse der Log-Komponente
Tabelle 9: Threshold-Sensitivity der Log-Komponente
Tabelle 10: Ablationsergebnisse auf Case-Ebene
Tabelle 11: False-Negative-Cases der Log-only-Komponente
Tabelle 12: Malicious Test-Szenarien: Recall
Tabelle 13: False-Positive-Analyse nach Szenario für Log-only

Abkürzungsverzeichnis

AP:	Average Precision
AUCPR:	Area Under the Precision-Recall Curve
CPU:	Central Processing Unit
CSV:	Comma-Separated Values
F1:	F1-Maß
FN:	False Negative
FP:	False Positive
FPR:	False-Positive-Rate
HTTP:	Hypertext Transfer Protocol
IP:	Internet Protocol
IPv4:	Internet Protocol Version 4
ISO:	International Organization for Standardization
KI:	Künstliche Intelligenz
ML:	Machine Learning
NaN:	Not a Number
PDF:	Portable Document Format
PID:	Process Identifier
PR:	Precision-Recall
ROC:	Receiver Operating Characteristic
ROC-AUC:	Area Under the Receiver Operating Characteristic Curve
SOC:	Security Operations Center
TF-IDF:	Term Frequency und Inverse Document Frequency
TN:	True Negative
TP:	True Positive
UUID:	Universally Unique Identifier
XGBoost:	Extreme Gradient Boosting
τ_{metric} :	Schwellenwert des Metrikmodells
τ_{log} :	Schwellenwert des Logmodells

1 Einleitung

1.1 Motivation

Automatisierte Anomalie-Erkennung ist ein zentrales Werkzeug im Betrieb großer Cloud-Infrastrukturen. Monitoring-Systeme werten kontinuierlich Systemmetriken aus und lösen bei auffälligen Abweichungen Alerts aus. In der Praxis führt dieser Ansatz jedoch zu einem hohen Aufkommen an Fehlalarmen (*False Positives*): Betriebsschwankungen, reguläre Lastspitzen oder geplante Batch-Jobs erzeugen Metrikanomalienmuster, die denen echter Angriffe oder Systemfehler ähneln, ohne es zu sein.

Dieses Phänomen ist in der Literatur als *Alert Fatigue* bekannt und stellt ein anerkanntes operatives Problem im Sicherheitsbetrieb dar. Es beeinträchtigt die Reaktionsfähigkeit von Analysten auf tatsächliche Vorfälle. Die Reduktion der False-Positive-Rate ist daher eine zentrale Anforderung an praxistaugliche Erkennungssysteme. Dabei soll die Erkennungsrate sicherheitsrelevanter Ereignisse nicht wesentlich sinken. Abschnitt 2.1 ordnet diesen Befund in die Literatur ein.

1.2 Problemstellung

Ein grundlegendes Problem metrischer Anomalie-Detektoren besteht im Zielkonflikt zwischen Recall und False-Positive-Rate. Um sicherzustellen, dass keine echten Angriffe unentdeckt bleiben (hoher Recall), muss der Entscheidungs-Threshold des Detektors niedrig gesetzt werden. Ein niedriger Threshold markiert jedoch zwangsläufig auch viele tatsächlich unbedenkliche Instanzen als auffällig, was die FPR in die Höhe treibt. Dieser Trade-off ist bei der Verwendung von wenigen, universell verfügbaren Metriken besonders ausgeprägt, da solche Features allein häufig keine ausreichende Trennschärfe zwischen malignen und benignen Fällen bieten.

Log-Dateien liefern einen ergänzenden, qualitativ anderen Informationstyp: Sie enthalten prozessuale Beschreibungen von Systemereignissen, die den Kontext einer Metrik-Auffälligkeit klären können [7]. Eine Lastspitze in der CPU-Auslastung kann sowohl bei einem malignen Prozess als auch bei einem legitimen Backup-Job auftreten. Der zugehörige Log-Eintrag hingegen unterscheidet beide Fälle in der Regel klar. Log-Daten eignen sich daher als nachgelagerte Verifikationsstufe: Alerts, die metrisch ausgelöst wurden, können auf Case-Ebene anhand ihres Log-Kontexts gefiltert werden.

Die Kombination beider Signalquellen zu einer hybriden Pipeline ist konzeptuell naheliegend. Die Pipeline besteht aus einem metrischen Trigger, gefolgt von einer log-basierten Verifikation. CloudAnoBench [16] stellt einen synthetischen Benchmark bereit, der für jeden überwachten Case sowohl Metriken als auch Log-Dateien bereitstellt und damit eine kontrollierte Evaluation eines solchen zweistufigen Ansatzes ermöglicht.

1.3 Zielsetzung und Forschungsfrage

Ziel dieser Arbeit ist die Entwicklung und Evaluation einer zweistufigen hybriden Anomalie-Erkennungspipeline auf CloudAnoBench. Der Schwerpunkt liegt auf einer methodisch kontrollierten Evaluation und der expliziten Diskussion der Grenzen des Ansatzes, einschließlich unerwarteter Befunde.

Die zentrale Forschungsfrage lautet:

Kann eine zweistufige hybride Pipeline, bestehend aus einem metrischen Anomalie-Detektor und einer log-basierten Nachfilterung, die False-Positive-Rate auf dem CloudAnoBench-Datensatz deutlich senken, ohne die Recall-Rate unter 0,95 zu drücken?

Die Arbeit untersucht diese Frage durch einen kontrollierten Ablations-Aufbau, der eine rein metrische Baseline, eine log-basierte Komponente sowie zwei Fusionsstrategien miteinander vergleicht, ein binäres AND-Gate und eine Soft-Fusion. Die Evaluation folgt dabei strikten Anforderungen an die Vermeidung von Datenleck und eine leakage-sichere Trainings-Test-Trennung.

Die Skalierbarkeitsbetrachtung dieser Arbeit ist als konzeptionelle Einordnung der modularen und grundsätzlich parallelisierbaren Pipeline zu verstehen. Eine empirische Untersuchung der Skalierbarkeit durch Lasttests, Durchsatz- oder Latenzmessungen beziehungsweise eine verteilte Implementierung ist nicht Gegenstand der Arbeit (vgl. Abschnitt 7.5).

1.4 Beiträge der Arbeit

Die vorliegende Arbeit leistet die folgenden konkreten Beiträge:

1. **Reproduzierbare Case-Level-Pipeline auf CloudAnoBench.** Es wird eine in Notebooks dokumentierte, reproduzierbare Pipeline entwickelt. Diese deckt alle Phasen ab, von der Datenaufbereitung über Case-Level-Feature-Engineering und Modelltraining bis hin zu Fusion und Evaluation auf CloudAnoBench. Insgesamt sichern 67 automatisierte Tests in 14 Testklassen zentrale Implementierungs- und Konsistenzbedingungen ab. Die Reproduzierbarkeit wird zusätzlich durch fixierte Seeds, gespeicherte Split-Zuordnungen, die dokumentierte Ausführungsreihenfolge und die spezifizierte Umgebung unterstützt. Der finale Testsplit wird mit `random_state=42` fixiert. Der interne Validierungsanteil wird ohne Zugriff auf das Testset über einen festen Seed aus einer vordefinierten Kandidatenliste gewählt.
2. **Case-Level-Metrik-Baseline mit XGBoost auf aggregierten Features.** Es wird eine metrische Baseline mit XGBoost [4] entwickelt, die direkt auf 25 aggregierten Case-Level-Features der fünf universellen Systemmetriken trainiert wird (Mittelwert, Maximum, Standardabweichung, 95. Perzentil und Trend je Metrik). Die Beschränkung auf die fünf universell verfügbaren Features ist eine methodisch begründete Entscheidung zur Vermeidung von Target Leakage durch strukturelle Fehlwerte.
3. **Log-basierte Verifikation mit Normalisierung, TF-IDF und logistischer Regression.** Es wird eine Case-Level-Komponente implementiert, die Log-Texte vor der TF-IDF-Vektorisierung durch eine regelbasierte Normalisierung synthetischer Artefakte bereinigt (Zeitstempel, IP-Adressen, PIDs, Ports, isolierte Zahlen werden durch generische Token ersetzt). Die logistische Regression auf dem

2. Grundlagen und verwandte Arbeiten

normalisierten TF-IDF-Raum wird als interpretierbare Baseline eingesetzt. Das Vokabular wird ausschließlich auf Trainingsdaten gefittet.

4. **Ablation der Fusionsstrategien einschließlich Degenerationsbefund.** Es werden zwei Fusionsstrategien evaluiert, AND-Gate und Soft-Fusion, und mit den Einzelkomponenten verglichen. Ein zentraler empirischer Befund ist die vollständige Degeneration des AND-Gates: Der metrische Detektor löst beim gewählten Threshold alle 253 Test-Cases aus (100 %), sodass das AND-Gate funktional äquivalent zur reinen Log-Filterung wird. Soft-Fusion verschlechtert sich gegenüber Log-only. Dieser Befund wird in Abschnitt 6.5 explizit diskutiert.
5. **Szenarioaufgelöste Fehleranalyse.** Es wird eine diagnostische Case-Level-Fehleranalyse durchgeführt, die False Negatives und False Positives nach Szenario aufschlüsselt, ohne Modelle neu zu trainieren oder Thresholds zu verändern. Die Analyse lokalisiert die Fehlerquellen und stützt die Interpretation der Hauptergebnisse (vgl. Abschnitt 6.4).

1.5 Aufbau der Arbeit

Abschnitt 2 legt die konzeptionellen und methodischen Grundlagen der Pipeline dar. Behandelt werden Cloud-Anomalieerkennung, unbalancierte Klassifikation und Precision-Recall-Metriken, XGBoost, TF-IDF, Data Leakage und gruppenbasierte Validierung sowie Klassifikatorfusion und Case-Level-Evaluation.

Abschnitt 3 beschreibt CloudAnoBench sowie die Datenaufbereitung und leakage-sichere Split-Strategie. Abschnitt 4 stellt die metrische Baseline (XGBoost auf aggregierten Case-Level-Features) vor. Abschnitt 5 beschreibt die log-basierte Verifikationskomponente (TF-IDF und logistische Regression). Abschnitt 6 präsentiert die hybride Fusion und die Ablationsstudie einschließlich der AND-Gate-Degeneration sowie der szenarioaufgelösten Fehleranalyse. Die Ergebnisse werden durch Tabellen und Abbildungen dokumentiert.

Abschnitt 7 diskutiert die Ergebnisse im Hinblick auf die Forschungsfrage, bewertet die Rolle der einzelnen Komponenten und benennt die methodischen Limitationen der Arbeit. Abschnitt 8 fasst die Befunde zusammen und gibt einen Ausblick auf mögliche Erweiterungen. Der Anhang enthält die zur Reproduzierbarkeit relevanten Artefakte sowie eine Dokumentation der KI-Nutzung.

2 Grundlagen und verwandte Arbeiten

2.1 Cloud-Anomalieerkennung und Alert Fatigue

Moderne Cloud-Umgebungen werden kontinuierlich über systemseitige Metriken wie CPU-Auslastung, Speicherverbrauch und Netzwerkdurchsatz überwacht. Anomalie-Detektoren werten diese Zeitreihendaten automatisiert aus und generieren Alerts bei auffälligen Abweichungen. In großen, dynamischen Infrastrukturen erzeugen solche Systeme jedoch regelmäßig eine hohe Zahl von Fehlalarmen, sogenannte False Positives, die Sicherheitsanalysten belasten und deren Reaktionsfähigkeit auf echte

Vorfälle verringern. Dieser Effekt ist in der Literatur als *Alert Fatigue* bekannt und stellt ein anerkanntes operatives Problem in Security Operations Centers dar [14]. Alahmadi et al. [1] belegen in einer qualitativen Studie, dass in real betriebenen SOC's ein erheblicher Teil der ausgelösten Sicherheitsalarme auf False Positives entfällt, was die Analysekapazität für tatsächliche Vorfälle systematisch einschränkt.

Die False-Positive-Rate ist dabei die zentrale operative Zielgröße: Sie beschreibt den Anteil tatsächlich negativer Instanzen, der fälschlicherweise als Alarm eingestuft wird. Eine hohe FPR verursacht nicht nur unnötigen Analyseaufwand, sondern kann im Extremfall dazu führen, dass echte Angriffe im Rauschen untergehen. Die Reduktion der FPR bei gleichzeitiger Aufrechterhaltung einer hohen Erkennungsrate (Recall) bildet daher die Forschungsfrage dieser Arbeit.

Als Evaluierungsgrundlage dient CloudAnoBench [16], ein synthetischer Benchmark, der Systemmetriken und Log-Dateien für Szenarien mit malignem, anomalem und normalem Systemverhalten bereitstellt. CloudAnoBench ist ein neuerer Benchmark und nicht als langjährig etablierter Goldstandard der Community zu verstehen. Die in dieser Arbeit erzielten Ergebnisse sind datensatzspezifisch und besitzen keine unmittelbare Allgemeingültigkeit.

2.2 Metrikbasierte und logbasierte Anomalieerkennung

Anomalieerkennung in Cloud-Systemen kann grundsätzlich auf zwei komplementären Informationsquellen operieren: Systemmetriken und Log-Dateien.

Systemmetriken sind numerische Zeitreihendaten, die den Ressourcenzustand eines Systems zu diskreten Zeitpunkten quantifizieren, etwa Prozessorauslastung, Speicherverbrauch oder Netzwerkdurchsatz. Sie sind gut strukturiert, hochfrequent verfügbar und lassen sich effizient mit tabellarischen Lernverfahren verarbeiten. Ihre Schwäche besteht darin, dass unterschiedliche Ereignistypen, etwa legitime Lastspitzen und Angriffsmuster, identische Metrikverläufe erzeugen können. Metriken allein erlauben in solchen Fällen keine zuverlässige Unterscheidung.

Log-Dateien hingegen enthalten textuelle Beschreibungen von Systemereignissen, Prozessaufrufen und Fehlerzuständen. Sie liefern einen prozessualen Kontext, der über die bloße Ressourcensicht hinausgeht. He et al. [7] zeigen in einer umfassenden Übersicht, dass automatisierte Log-Analyse ein etabliertes Werkzeug im Reliability Engineering ist: Textmuster in Log-Dateien können charakteristische Signaturen für unterschiedliche Ereigniskategorien tragen, die in den Metriken nicht sichtbar sind.

Die vorliegende Arbeit verfolgt einen zweistufigen Ansatz: Ein metrischer Detektor identifiziert zunächst potenzielle Auffälligkeiten auf Basis von Ressourcensignalen. Eine log-basierte Komponente verifiziert anschließend auf Case-Ebene, ob der Log-Kontext den Alert stützt. Beide Komponenten sind methodisch aufeinander abgestimmt und werden in Abschnitt 3 im Detail beschrieben.

2.3 Unbalancierte Klassifikation und Precision-Recall-Metriken

Anomalie-Erkennungsprobleme sind typischerweise stark unbalanciert: Positive Instanzen (Angriffe, Fehler) sind weit seltener als negative (Normalbetrieb). He und Garcia [6] zeigen, dass in solchen Szenarien standardmäßige Klassifikationsmetriken wie Accuracy irreführend sind, da sie von der Mehrheitsklasse dominiert werden.

Die für diese Arbeit relevanten Metriken sind:

Precision $P = \frac{TP}{TP+FP}$: Anteil der korrekt als positiv klassifizierten Instanzen an allen als positiv gekennzeichneten.

Recall $R = \frac{TP}{TP+FN}$: Anteil der tatsächlich positiven Instanzen, die erkannt werden. In dieser Arbeit dient $R \geq 0,95$ als Zielkriterium für die Threshold-Wahl auf dem Validierungsset. Ob dieser Wert im unabhängigen Testsplitt erreicht wird, ist eine empirische Frage.

False-Positive-Rate $FPR = \frac{FP}{FP+TN}$: Anteil der negativen Instanzen, die fälschlicherweise als positiv eingestuft werden. Dies ist die zentrale operative Zielgröße.

Average Precision AP: Fläche unter der Precision-Recall-Kurve, berechnet als gewichteter Mittelwert der Precision-Werte über alle Recall-Stufen. AP fasst die Klassifikatorleistung über alle möglichen Thresholds zusammen.

Davis und Goadrich [5] zeigen, dass die PR-Kurve gegenüber der ROC-Kurve bei unbalancierten Problemen informativer ist: Während der ROC-AUC die Leistung im negativen Regime mitunter überschätzt, macht die PR-Kurve Schwächen bei der Behandlung der Minderheitsklasse direkt sichtbar. Saito und Rehmsmeier [13] bestätigen dies und empfehlen PR-Kurven explizit für stark unbalancierte Klassifikationsprobleme. Diese Befunde motivieren in dieser Arbeit die Verwendung von AUCPR als primäre Optimierungsmetrik sowie die Fokussierung auf FPR und AP in der Evaluation.

2.4 XGBoost für tabellarische Metrikdaten

Als metrischer Basis-Detektor wird in dieser Arbeit XGBoost [4] eingesetzt. XGBoost implementiert Gradient Tree Boosting und kann den Split-Finding-Algorithmus durch eine histogrammbasierte Methode approximieren, die den Trainingsaufwand gegenüber einer exakten Split-Suche deutlich reduziert. Für tabellarisch strukturierte Daten mit numerischen Features und heterogenen Skalen stellt XGBoost ein in der Literatur breit erprobtes Verfahren dar.

Relevant für diese Arbeit sind zudem die Möglichkeit, die Minderheitsklasse im unbalancierten Setting stärker zu gewichten, sowie die Möglichkeit, AUCPR direkt als Optimierungsmetrik zu verwenden.

XGBoost wird hier nicht als grundsätzlich überlegenes Verfahren positioniert, sondern als solide, reproduzierbare Baseline für tabellarische Klassifikation. Es ist nicht Ziel dieser Arbeit, das metrische Modell zu optimieren. Vielmehr soll seine Leistungsgrenze die Notwendigkeit einer ergänzenden Log-Komponente motivieren.

2.5 TF-IDF und logistische Regression für Logdaten

Die log-basierte Komponente dieser Arbeit nutzt eine Kombination aus TF-IDF-Vektorisierung und logistischer Regression. TF-IDF gewichtet Terme proportional zu ihrer Häufigkeit im Dokument, normiert jedoch durch ihre Häufigkeit über alle Dokumente. Häufige, wenig diskriminierende Terme, in Log-Dateien etwa routinemäßige Statusmeldungen, erhalten dadurch geringere Gewichte. Seltene, charakteristische Terme werden hervorgehoben.

Auf der TF-IDF-Matrix wird eine logistische Regression trainiert. Lineare Modelle auf sparse hochdimensionalen Textrepräsentationen sind für diese Problemklasse eine etablierte Wahl. Der Klassifikator wird in dieser Arbeit mit scikit-learn [11] umgesetzt. Die logistische Regression liefert Scores, die in der nachgelagerten Fusionsstufe direkt weiterverwendet werden. Eine explizite Wahrscheinlichkeitskalibrierung (z. B. Platt Scaling) wurde nicht durchgeführt, sondern die Ausgaben wurden als Modellscores interpretiert.

Die Entscheidung für TF-IDF und logistische Regression folgt dem Prinzip einer interpretierbaren und effizient trainierbaren Baseline. Fortgeschrittenere Ansätze zur Log-Analyse, etwa struktur-bewusste Log-Parser, vortrainierte Sprachmodelle oder Embedding-Methoden, lagen außerhalb des festgelegten Untersuchungsrahmens dieser Arbeit. Sie stellen eine naheliegende Erweiterung dar, waren aber nicht Gegenstand der vorliegenden Evaluation.

2.6 Data Leakage und gruppenbasierte Validierung

Eine korrekte Evaluation maschineller Lernverfahren setzt voraus, dass keine Information aus dem Testset in den Trainingsprozess einfließt. Kaufman et al. [9] zeigen, dass Informationen aus dem Evaluationsanteil nicht in den Lernprozess einfließen dürfen, da dies zu optimistischen Leistungsschätzungen führen kann. Auf die vorliegende Pipeline übertragen betrifft dies beispielsweise eine globale Skalierung oder Imputation vor der Datenaufteilung.

Für diese Arbeit ist eine spezifische Form von Leakage besonders relevant: *Schema-Leakage* durch strukturelle Fehlwerte. Werden verschiedene Klassen mit systematisch unterschiedlichen Feature-Schemata erfasst, entstehen typspezifische NaN-Muster, die ein Modell implizit zur Bestimmung der Klassenzugehörigkeit nutzen kann, noch bevor inhaltlich relevante Signale verarbeitet werden. Dies entspricht dem von Kaufman et al. beschriebenen Prinzip eines unzulässigen Merkmals, da die Feature-Struktur Informationen über die Zielvariable trägt und dadurch eine valide Evaluation gefährden kann [9]. Die methodische Konsequenz ist die Beschränkung der Modellierung auf Features, die in allen Klassen vollständig verfügbar sind.

Ein zweites Leakage-Risiko ergibt sich aus der Datensatzstruktur: Beobachtungen innerhalb desselben Szenarios sind strukturell korreliert. Eine zufällige Aufteilung in Trainings- und Testmenge ohne Berücksichtigung dieser Gruppen würde Szenarien auf beiden Seiten des Splits erlauben und die Trennleistung systematisch überschätzen. Roberts et al. [12] zeigen, dass bei Daten mit gruppenweise korrelierten Beobachtungen eine gruppenweise Partitionierung methodisch empfehlenswert ist. In dieser Arbeit

wird `GroupShuffleSplit` verwendet, das sicherstellt, dass kein Szenario gleichzeitig in Trainings- und Testmenge vertreten ist.

2.7 Klassifikatorfusion und Case-Level-Evaluation

Die Kombination mehrerer Klassifikatoren zu einem gemeinsamen Entscheidungssystem ist ein etabliertes Forschungsfeld. Kittler et al. [10] leiten mehrere Fusionsregeln, darunter Produkt-Regel, Summen-Regel und Mehrheitsentscheid, als unterschiedliche Näherungen innerhalb eines gemeinsamen Bayes-Rahmens her und zeigen in ihren Experimenten eine besondere Robustheit der Summenregel.

In dieser Arbeit wird die Fusionsentscheidung direkt auf Case-Ebene getroffen. Beide Komponenten liefern pro Case c einen Score: $\hat{p}_{\text{metric}}(c)$ aus dem metrischen XGBoost-Klassifikator, der auf 25 Case-Level-Aggregaten der fünf universellen Systemmetriken trainiert wird (je Metrik: Mittelwert, Maximum, Standardabweichung, 95. Perzentil und linearer Trend), sowie $\hat{p}_{\log}(c)$ aus der log-basierten logistischen Regression auf dem normalisierten TF-IDF-Raum. Beide Komponenten arbeiten direkt auf Case-Ebene. Ein zwischengeschalteter Row-Level-Aggregationsschritt entfällt.

Zwei Fusionsstrategien werden untersucht. Das **AND-Gate** stellt eine konservative Strategie dar: Ein Alert wird nur ausgegeben, wenn *beide* Komponenten positiv entscheiden:

$$\hat{y}_{\text{AND}}(c) = \mathbf{1}[\hat{p}_{\text{metric}}(c) \geq \tau_{\text{metric}} \wedge \hat{p}_{\log}(c) \geq \tau_{\log}]$$

Die **Soft-Fusion** kombiniert beide Scores als Produkt:

$$s_{\text{fusion}}(c) = \hat{p}_{\text{metric}}(c) \times \hat{p}_{\log}(c)$$

Da beide Faktoren im Intervall $[0, 1]$ liegen, gilt dies auch für $s_{\text{fusion}}(c)$. Kittler et al. [10] zeigen, dass die Produktregel unter der Annahme unabhängiger Klassifikatoren asymptotisch optimal ist. Wie gut diese Annahme auf den vorliegenden Daten zutrifft, ist eine empirische Frage. Die Thresholds τ_{metric} und $\tau_{\log}$ werden ausschließlich auf dem Validierungsset bestimmt. Das Testset wird nur zur finalen Evaluation verwendet. Ob die AND-Gate-Strategie gegenüber einer Einzelkomponente einen Mehrwert liefert, wird in Abschnitt 6.3 untersucht.

3 Datensatz und Datenaufbereitung

3.1 CloudAnoBench und Datenbasis

Grundlage der Untersuchung ist CloudAnoBench [16], ein synthetischer Benchmark zur multimodalen Anomalieerkennung in Cloud-Umgebungen. Der Datensatz stellt für jede Beobachtungseinheit sowohl Zeitreihendaten von Systemmetriken als auch zugehörige Log-Dateien bereit und eignet sich damit explizit für eine hybride Analyse, wie sie in dieser Arbeit untersucht wird. Es sei ausdrücklich darauf hingewiesen, dass CloudAnoBench ein neuerer Benchmark ist, der noch nicht als etablierter Goldstandard der Community gilt. Die Ergebnisse dieser Arbeit sind daher im Kontext dieses spezifischen Datensatzes zu interpretieren.

Der Datensatz umfasst drei inhaltlich distinkte Typen: *mali* (maliciously anomalous), *anom* (non-malicious anomalous) und *norm* (normal). Die Unterscheidung ist für die binäre Klassifikationsaufgabe relevant, da nur *mali*-Instanzen als positiv (Label 1) und *anom* sowie *norm* als negativ (Label 0) kodiert werden. Anom-Fälle stellen dabei nicht-maliziöse technische Anomalien dar, die von einem Sicherheitsdetektor korrekt als unbedenklich eingestuft werden sollen. Das CloudAnoBench-Paper hebt insbesondere die operativen Kosten von Fehlalarmen bei normalen Fällen hervor [16]. Für die vorliegende Arbeit ist daher die Reduktion von Fehlalarmen bei nicht-maliziösen Fällen von besonderer Bedeutung. Norm-Cases repräsentieren planmäßigen Betrieb, der aufgrund ressourcenintensiver, aber nicht-maliziöser Aktivitäten, etwa Backup- oder Batch-Prozesse, metrisch auffällig wirken kann. Norm-Cases sind daher eine besonders relevante Quelle von Fehlalarmen. In der Evaluation werden sie über eine separate FPR_{norm} ausgewertet.

Tabelle 1 fasst den Umfang der Datenbasis nach Typ auf Zeilen- und Case-Ebene zusammen.

Tabelle 1: Datenbasis: Zeilen und Cases nach Typ

Typ	Zeilen (roh)	Zeilen (bereinigt)	Cases	Label
mali	19.385	19.384	223	positiv (1)
anom	41.400	41.400	460	negativ (0)
norm	51.208	51.208	569	negativ (0)
Gesamt	111.993	111.992	1.252	

Lediglich eine einzige Zeile in *mali* wurde beim Bereinigungsschritt entfernt (0,01 %), was die Datenqualität als insgesamt hoch ausweist. Allen 1.252 Cases ist genau eine Log-Datei zugeordnet. Kein Fall weist eine fehlende Log-Datei auf. Im für diese Arbeit verwendeten Hugging-Face-Export ergeben sich aus der Kombination von `dataset_type` und `scenario_id` insgesamt 45 Gruppen: 17 *anom*-, 12 *mali*- und 16 *norm*-Gruppen.

3.2 Datenqualität und Preprocessing

Die gesamte experimentelle Pipeline ist in Python 3.12 implementiert. Weitere Angaben zur Implementierung und Reproduzierbarkeit befinden sich in Anhang A.

Die Rohdaten liegen im CSV-Format vor und weisen zwei typische Qualitätsprobleme auf. Erstens enthalten einige Felder residuale Anführungszeichen aus einer überlappenden CSV-Kodierung (Parameter `quoting=3` beim Einlesen), die durch eine explizite Zeichenkettenbereinigung entfernt werden. Zweitens sind numerische Felder in einigen Dateien als Zeichenketten kodiert und müssen in Fließkommazahlen überführt werden.

3. Datensatz und Datenaufbereitung

Die leakage-sichere Imputation fehlender Werte folgt einer festen Reihenfolge über mehrere Phasen. In Phase 1 (Notebook 01) wird das Schema harmonisiert und die numerische Konvertierung durchgeführt. Fehlende Werte bleiben dabei als NaN erhalten. Die gespeicherten Parquet-Artefakte unter `data/processed/` können deshalb weiterhin NaN-Werte enthalten. Der Ordner `data/` ist nicht Bestandteil des öffentlichen Repositories und muss lokal bereitgestellt werden. In Phase 2 (Notebook 02) werden zunächst die fixierten Split-Zuordnungen angewendet. Für die fünf universellen Zeilenmetriken werden die Mediane ausschließlich aus den Trainingszeilen gemeinsam über `mali`, `anom` und `norm` bestimmt und in `data/processed/train_medians.json` gespeichert. Dieselben train-basierten Werte werden unverändert auf Train, Validation und Test angewendet. Erst danach werden die 25 Case-Level-Aggregate gebildet. Falls auf Case-Ebene noch fehlende Werte verbleiben, werden ergänzende Featuremediane ausschließlich aus `X_train` bestimmt und unverändert auf Validation und Test angewendet. Eine nach `dataset_type` getrennte Imputation findet nicht statt. Diese Trennung schützt vor einem verbreiteten Fehler bei der Vorverarbeitung, der in der Literatur als *Data Leakage* bezeichnet wird [9].

Die Wahl des Medians gegenüber dem arithmetischen Mittel ist durch die Verteilung der Metriken motiviert: Systemmetriken wie `net_out` zeigen häufig eine rechtsschiefe Verteilung mit einzelnen Ausreißerspitzen, gegen die der Median robust ist [8].

3.3 Kanonisches Feature-Set und universelle Features

Im für diese Arbeit verwendeten Hugging-Face-Export zeigt sich eine typspezifisch unterschiedliche Belegung der kanonischen Metrikspalten. Um eine einheitliche Datengrundlage zu schaffen, wird eine *kanonische Feature-Matrix* mit 34 Features aufgebaut, die die Vereinigung aller je gemessenen Metriken darstellt. Fehlende Spalten werden mit NaN aufgefüllt.

Von diesen 34 Features sind lediglich fünf in allen drei Datensatztypen vorhanden:

```
cpu_usage, mem_usage, disk_io, net_in, net_out
```

Die verbleibenden 29 Features sind typspezifisch und strukturell in mindestens einem Typ vollständig als NaN belegt: In `mali` fehlen 28 von 34 Features strukturell, in `anom` 23 und in `norm` lediglich sechs. Das Ausmaß dieser strukturellen Fehlwerte ist kein Zufallsartefakt, sondern spiegelt direkt die unterschiedlichen Erhebungsmethoden je Datensatztyp wider.

Abbildung 1 zeigt die Verteilung der fünf universellen Features getrennt nach den drei Datensatztypen anom, mali und norm.

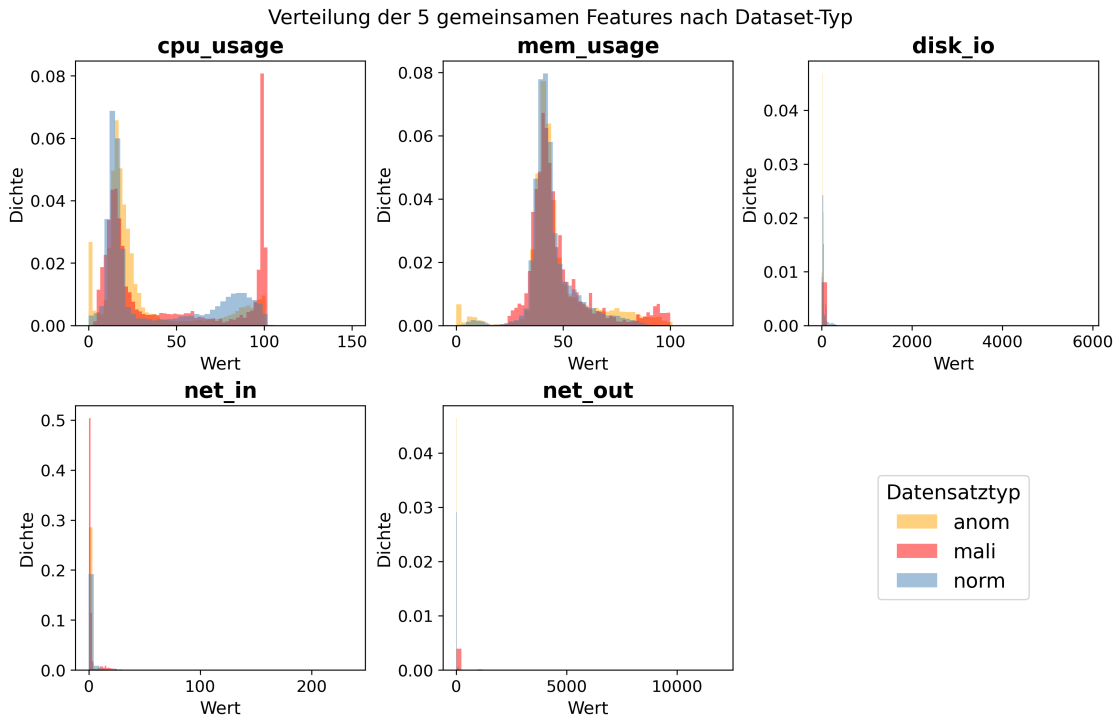


Abbildung 1: Verteilung der fünf universellen Features nach Datensatztyp

3.4 Leakage-Vermeidung und Split-Strategie

Für eine leakage-sichere Evaluation werden die Daten in drei disjunkte Teilmengen aufgeteilt. Als Gruppenkennung dient die Kombination aus `dataset_type` und `scenario_id` (`group_id`), sodass kein Szenario in mehr als einer Teilmenge erscheint. Die Aufteilung erfolgt in zwei Stufen mit `GroupShuffleSplit` [12]: In einem ersten äußeren Schritt werden ca. 20 % der Gruppen als Testset abgetrennt. In einem zweiten inneren Schritt werden aus dem verbleibenden Pool ca. 20 % als Validierungsset reserviert. Das Trainingsset dient ausschließlich dem Modelltraining und der Imputation. Das Validierungsset wird für Early Stopping und die Threshold-Wahl verwendet, und das Testset wird ausschließlich zur finalen Evaluation eingesetzt. Die Nicht-Überlappung aller drei Splits nach Gruppenkennung wurde artefaktseitig verifiziert (`docs/split_assignments.csv`). Beobachtungen innerhalb desselben Szenarios sind strukturell korreliert, sodass eine zufällige Aufteilung ohne Berücksichtigung dieser Gruppenstruktur die Trennleistung im Test systematisch überschätzen würde.

4. Metrikbasierte Baseline

Tabelle 2 fasst die resultierenden Teilmengen zusammen. Der finale Testsplit wird mit `random_state=42` fixiert. Das Klassenungleichgewicht im Trainingsset beträgt 5,50:1 (negativ zu positiv, 671 vs. 122 Cases).

Tabelle 2: Gruppenbasierter Train/Validation/Test-Split

Split	Cases	Gruppen	Positiv	Negativ
Train	793	28	122	671
Validation	206	8	53	153
Test	253	9	48	205
Gesamt	1.252	45	223	1.029

4 Metrikbasierte Baseline

4.1 Modellwahl und Hyperparameter

Als Baseline-Modell für den metrikbasierten Anomalie-Detektor wird XGBoost [4] über die Klasse `XGBClassifier` eingesetzt. XGBoost implementiert Gradient Boosting über Entscheidungsbäume und ist durch die `hist`-Methode (histogrammbasiertes Split-Finding) auch für größere Datensätze effizient anwendbar. Für tabellarisch strukturierte Daten mit gemischten Skalen stellt XGBoost eine gut validierte Wahl dar.

Als Optimierungsmetrik wird AUCPR gewählt. Bei stark unbalancierten Klassifikationsproblemen ist AUCPR gegenüber dem ROC-AUC informativer, da Letzterer die Leistung im negativen Regime übergewichtet [5, 13]. Das vorliegende Klassenungleichgewicht im Trainingsset von annähernd 5,50:1 stellt eine moderate Klassenunwucht dar [6].

Der Klasse `XGBClassifier` wird über den Parameter `scale_pos_weight` eine Gewichtung der positiven Minderheitsklasse übergeben. Der Wert von 5,50 wird ausschließlich aus der Trainingsverteilung berechnet (671 negative vs. 122 positive Trainingsfälle). Early Stopping ist mit 20 Runden aktiviert. Die Überwachungsmetrik wird dabei ausschließlich auf dem Validierungsset berechnet. Der Operating-Threshold wird als höchster Validierungsschwellenwert gewählt, der auf dem Validierungsset eine Recall-Rate von mindestens 0,95 erzielt. Die so bestimmte Schwelle wird unverändert auf das Testset angewendet.

Tabelle 3 fasst die für die Metrik-Baseline verwendete Hyperparameterkonfiguration des XGBoost-Modells zusammen.

Tabelle 3: XGBoost-Hyperparameter

Parameter	Wert
<code>tree_method</code>	<code>hist</code>
<code>eval_metric</code>	<code>aucpr</code>
<code>scale_pos_weight</code>	5,50
<code>random_state</code>	42
Early Stopping	aktiv (Überwachung auf Validierungsset)

4.2 Schema-Leakage, Feature-Reduktion und Case-Level-Aggregation

Eine direkte Verwendung der vollständigen kanonischen 34-Feature-Matrix würde ein gravierendes Datenleck-Problem erzeugen. Die 29 typspezifischen Features sind strukturell in bestimmten Typen als NaN belegt: Ein Modell, das auf dieser Matrix trainiert, kann das bloße Muster fehlender Werte zur Bestimmung des Datensatztyps und damit der Klassenzugehörigkeit nutzen, noch bevor es ein inhaltliches Signal verarbeitet hat. Dies entspricht dem von Kaufman et al. beschriebenen Konzept eines informationsleckenden Merkmals, da die Feature-Struktur die Zielklasse implizit enkodiert [9].

Die Metrik-Baseline wird daher ausschließlich auf den fünf universellen Features trainiert und evaluiert (vgl. Abschnitt 3.3). Diese sind in allen drei Datensatztypen vorhanden und tragen kein typspezifisches Strukturmuster.

Um Zeitreiheninformationen auf Case-Ebene nutzbar zu machen, werden die Rohmessreihen jeder Case pro Metrik zu fünf statistischen Aggregaten verdichtet: Mittelwert (mean), Maximum (max), Standardabweichung (std), 95. Perzentil (p95) und linearer Trend (slope). Dies ergibt eine Feature-Matrix mit $5 \times 5 = 25$ Case-Level-Features. Das XGBoost-Modell wird direkt auf dieser Case-Level-Feature-Matrix trainiert. Ein separater Row-Level-Klassifikationsschritt entfällt. Fehlende Feature-Werte werden durch den auf dem Trainingsset berechneten Median imputiert (kein Leakage).

4.3 Ergebnisse der Metrik-Baseline

Tabelle 4 stellt die Testset-Ergebnisse der Metrik-Baseline (XGBoost auf 25 Case-Level-Features, Testset $n = 253$) auf Case-Ebene dar.

Tabelle 4: Ergebnisse der Metrik-Baseline

Metrik	Wert
AP	0,2652
Threshold ($\text{Val-Recall}_{\text{mali}} \geq 0,95$)	0,0258
$\text{Recall}_{\text{mali}} @ \text{Threshold}$	1,0000
$\text{Precision} @ \text{Threshold}$	0,1897
$\text{F1-Ma\ss} @ \text{Threshold}$	0,3189
$\text{FPR}_{\text{total}}$	1,0000 (205 / 205 Negative)
FPR_{anom}	1,0000 (76 / 76)
FPR_{norm}	1,0000 (129 / 129)
TP / FP / TN / FN	48 / 205 / 0 / 0

Der XGBoost-Detektor erkennt auf Case-Ebene alle 48 positiven Test-Cases ($\text{Recall}_{\text{mali}} = 1,0000$), klassifiziert aber gleichzeitig alle 205 negativen Cases als positiv ($\text{FPR}_{\text{total}} = 1,0000$, $\text{TN} = 0$). Die Metrik-Komponente ist auf diesem Benchmark *nicht selektiv*: Sie kann nicht zwischen mali-, anom- und norm-Cases unterscheiden, weil viele technische Anomalien und Normalszenarien ähnlich hohe Metrik-Aggregate wie Malware-Aktivitäten aufweisen. Der niedrige Threshold von 0,0258 (bestimmt auf dem Validierungsset) markiert alle Test-Cases als metrisch auffällig.

4. Metrikbasierte Baseline

Die Metrik-Baseline ist auf Case-Ebene zwar vollständig sensitiv ($\text{Recall}_{\text{mali}} = 1,0000$), aber nicht selektiv ($\text{FPR}_{\text{total}} = 1,0000$). Dieser Befund motiviert die in Abschnitt 5 beschriebene log-basierte Komponente.

Abbildung 2 zeigt die zugehörige Precision-Recall-Kurve.

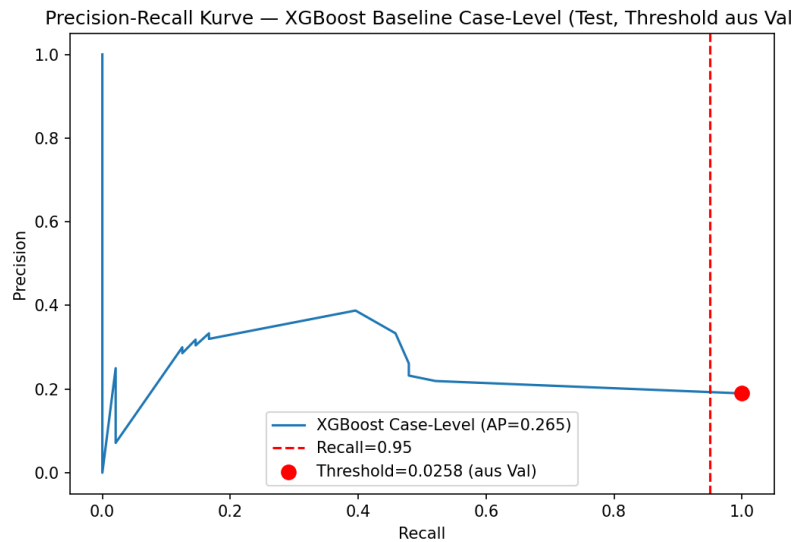


Abbildung 2: Precision-Recall-Kurve der Metrik-Baseline

Die Kurve bezieht sich auf die Case-Ebene und erreicht eine AP von 0,2652.

Der markierte Punkt entspricht dem auf dem Validierungsset bestimmten Threshold von 0,0258 bei $\text{Recall}_{\text{mali}} = 1,0000$.

Abbildung 3 visualisiert die Feature-Importances des trainierten Modells nach dem Gain-Kriterium auf den 25 Case-Level-Features (fünf Metriken mit je fünf Aggregationen).

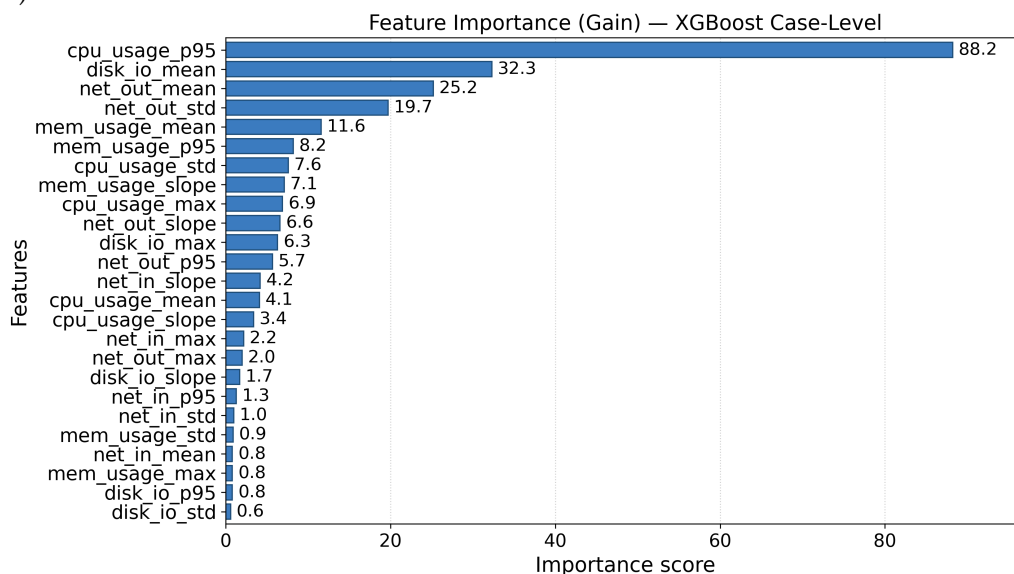


Abbildung 3: Feature-Importances des XGBoost-Modells

5 Log-basierte Verifikation

5.1 Log-Daten auf Case-Ebene

Für jeden der 1.252 Cases liegt eine zugehörige Log-Datei vor. Die Log-Analyse operiert analog zur metrischen Analyse auf Case-Ebene: Jede Log-Datei wird als ein einzelnes Dokument behandelt, dem genau ein binäres Label zugewiesen ist. Log-Daten enthalten qualitative Textinformationen über Systemereignisse, Fehlermeldungen und Prozessabläufe, die Metriken allein nicht abbilden können. He et al. [7] zeigen in ihrer Übersicht, dass automatisierte Log-Analyse ein etabliertes Werkzeug im Reliability Engineering ist und textbasierte Merkmale relevante Signale für die Anomalieerkennung liefern.

Von 1.252 Cases des Gesamtdatensatzes entfallen nach dem gruppenbasierten dreistufigen Split 793 Fälle auf das Trainingsset, 206 auf das Validierungsset und 253 auf das Testset. Im Testset befinden sich 48 positive (mali), 76 anom und 129 norm-Cases.

5.2 Log-Normalisierung

Bevor die Log-Texte vektorisiert werden, werden synthetische Artefakte durch generische Token-Platzhalter ersetzt. Dieser Schritt ist deterministisch und regelbasiert. Er verursacht kein Datenleck, da er unabhängig von den Labels ist und auf alle Splits gleichartig angewendet wird. Die Motivation dieser Normalisierung besteht darin, dass synthetische Artefakte wie Zeitstempel und IP-Adressen in einem Benchmarkdatensatz zufällig variieren und kein semantisches Signal tragen. Das TF-IDF-Modell soll stattdessen auf dem inhaltlich relevanten Vokabular trainiert werden. Tabelle 5 gibt einen Überblick über die ersetzten Muster und die zugehörigen Token-Platzhalter.

Tabelle 5: Regelbasierte Log-Normalisierung

Muster	Token	Beispiel
UUIDs	TOKEN_ID	550e8400-... → TOKEN_ID
Nginx-Zeitstempel	TOKEN_TIME	[20/Aug/2025:08:01:10 +0000] → TOKEN_TIME
Syslog-Zeitstempel	TOKEN_TIME	Aug 15 10:00:05 → TOKEN_TIME
ISO-Zeitstempel (mit Zeit)	TOKEN_TIME	2024-01-12T13:45:22Z → TOKEN_TIME
ISO-Datum	TOKEN_DATE	2024-01-12 → TOKEN_DATE
Uhrzeiten	TOKEN_TIME	13:45:22 → TOKEN_TIME
IPv4-Adressen	TOKEN_IP	192.168.1.10 → TOKEN_IP
PIDs (in [])	TOKEN_PID	sshd[11532] → sshd[TOKEN_PID]
Ports	TOKEN_PORT	port 54321 → port TOKEN_PORT
Isolierte Zahlen	TOKEN_NUM	retry 3 → retry TOKEN_NUM

Sicherheitsrelevante semantische Information bleibt erhalten: Prozessnamen (sshd, CRON, wget, curl, bash), Aktionen (Accepted, Failed, CMD), Pfade, Domains und Protokollbezeichner werden nicht verändert. Es sei darauf hingewiesen, dass einige dieser Terme, etwa sshd, cron oder bash, in regulärem Systembetrieb generisch auftreten können. Sie sind Teil der vordefinierten Security-Keyword-Liste, nicht per se Angriffshinweise.

5.3 TF-IDF-Vektorisierung

Die Textrepräsentation erfolgt mit TF-IDF. TF-IDF gewichtet Terme nach ihrer Auftretenshäufigkeit im Dokument, normiert dabei jedoch durch die Häufigkeit über alle Dokumente hinweg, sodass häufige, wenig diskriminierende Terme abgewertet werden. TF-IDF stellt eine interpretierbare und effizient trainierbare Baseline dar. Fortgeschrittenere Ansätze wie Log-Parser oder Embedding-Methoden lagen außerhalb des festgelegten Untersuchungsrahmens dieser Arbeit und bilden eine naheliegende Erweiterung. He et al. [7] beschreiben die automatisierte Analyse von Logdaten als etablierten Ansatz zur Erkennung und Diagnose von Systemereignissen. In dieser Arbeit wird TF-IDF eingesetzt, um häufige generische Terme wie regelmäßige Statusmeldungen geringer zu gewichten. Tabelle 6 fasst die verwendete Konfiguration der TF-IDF-Vektorisierung zusammen.

Tabelle 6: TF-IDF-Konfiguration

Parameter	Wert
max_features	10.000
ngram_range	(1, 2)
sublinear_tf	True
min_df	2
Fit	ausschließlich auf Trainingsfälle

Die `TfidfVectorizer`-Instanz wird ausschließlich auf den 793 Trainingsfällen gefittet. Weder das Validierungs- noch das Test-Vokabular wird einbezogen [9]. Die Verwendung von Bigrammen erlaubt die Erfassung kurzer Wortfolgen, die in Log-Meldungen häufig als feste Phrasen auftreten. Der Parameter `ngram_range` ist auf (1, 2) gesetzt. Mit `sublinear_tf=True` wird die rohe Termfrequenz durch ihren Logarithmus ersetzt, um sehr häufige Terme nicht unverhältnismäßig zu gewichten. Das resultierende Vokabular umfasst 10.000 Terme. Die Trainings-, Validierungs- und Testmatrix haben die Dimension (793×10.000) , (206×10.000) bzw. (253×10.000) .

5.4 Logistische Regression

Auf der TF-IDF-Matrix wird eine logistische Regression trainiert. Lineare Klassifikatoren auf sparse hochdimensionalen Textmatrizen sind für diese Problemklasse eine gut erprobte Wahl. Die Implementierung erfolgt über `LogisticRegression` aus `scikit-learn` [11]. Der `lbfgs`-Solver ist auf mittelgroßen Datensätzen numerisch stabil und effizient einsetzbar. Das Klassenungleichgewicht (Trainingsverhältnis $\approx 5,50:1$) wird durch `class_weight='balanced'` ausgeglichen, das die Verlustfunktion proportional zu den Klassengrößen gewichtet. Tabelle 7 fasst die Hyperparameter der logistischen Regression zusammen.

Tabelle 7: Hyperparameter der logistischen Regression

Parameter	Wert
class_weight	balanced
solver	lbfgs
C	1,0
max_iter	1.000

Abbildung 4 visualisiert die 20 einflussreichsten Terme des Modells nach Koeffizientengröße, getrennt nach positiv und negativ gewichteten Termen. Positiv gewichtete Terme sind stärker mit malignen Cases assoziiert, während negativ gewichtete Terme häufiger in benignen Cases auftreten.

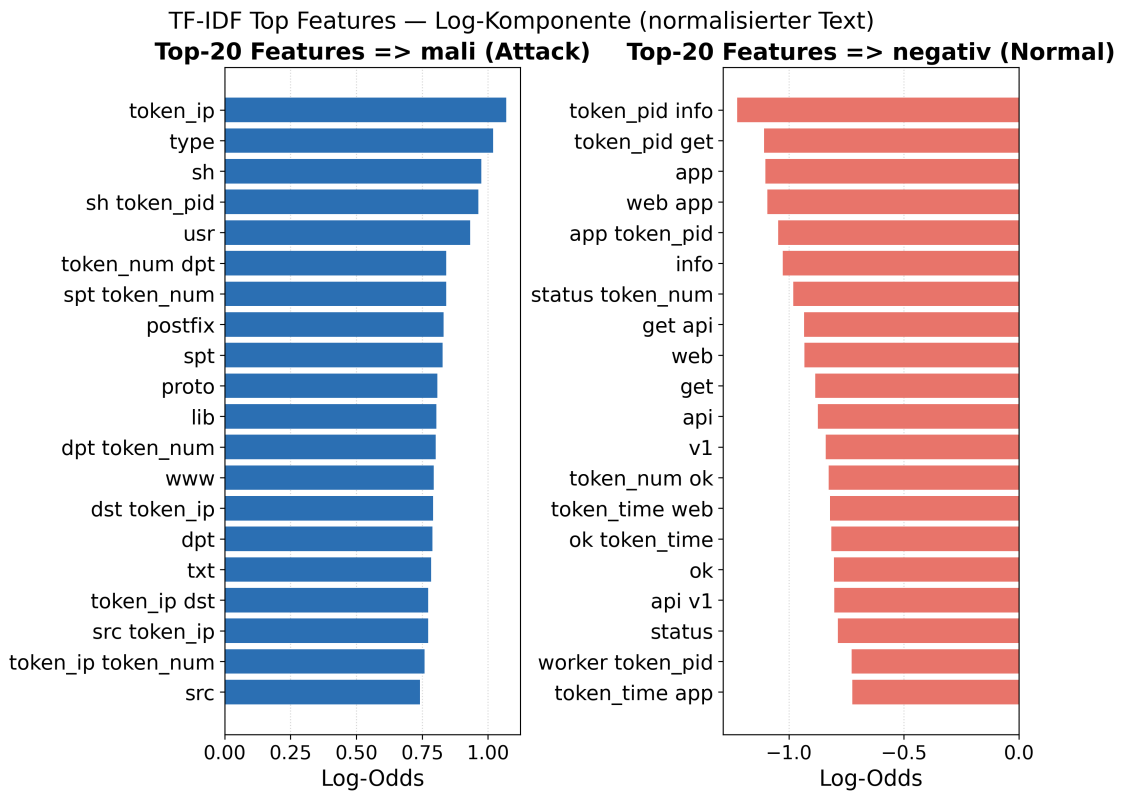


Abbildung 4: Top-20 TF-IDF-Terme nach Koeffizientengröße

Der Operating-Threshold der Log-Komponente wird auf dem Validierungsset bestimmt: Es wird der höchste Validierungsschwellenwert gewählt, der auf dem Validierungsset eine $\text{Recall}_{\text{mali}}$ -Rate von mindestens 0,95 erzielt. Die so bestimmte Schwelle (0,164) wird anschließend unverändert auf das Testset angewendet.

5.5 Ergebnisse der Log-Komponente

Tabelle 8 stellt die Testset-Ergebnisse der Log-Komponente (TF-IDF und logistische Regression auf normalisierten Log-Texten, Testset $n = 253$) auf Case-Ebene dar.

Tabelle 8: Ergebnisse der Log-Komponente

Metrik	Wert
AP	0,6548
Threshold (Val-Recall _{mali} $\geq 0,95$)	0,1641
Recall _{mali} @ Threshold	0,9375
Precision @ Threshold	0,4639
F1-Maß @ Threshold	0,6207
FPR _{total}	0,2537 (52 / 205 Negative)
FPR _{anom}	0,2368 (18 / 76)
FPR _{norm}	0,2636 (34 / 129)
TP / FP / TN / FN	45 / 52 / 153 / 3

Gegenüber der nicht-selektiven Metrik-Baseline (FPR_{total} = 1,0000) sinkt die FPR auf 0,2537. Die Log-Komponente ist die einzige Variante mit echter Selektivität. Der Test-Recall_{mali} beträgt 0,9375 (45 von 48 positiven Cases erkannt, 3 FN), womit die angestrebte Schwelle von 0,95 knapp verfehlt wird. Getrennt nach Negativtypen ergeben sich FPR_{anom} = 0,2368 und FPR_{norm} = 0,2636. Ein direkter AP-Vergleich mit der Metrik-Baseline ist nun auf gleicher Granularität (Case-Ebene, $n = 253$) möglich.

Abbildung 5 zeigt die zugehörige PR-Kurve auf Case-Ebene, die eine AP von 0,6548 erreicht. Der markierte Punkt entspricht dem auf dem Validierungsset bestimmten Threshold von 0,164 bei Recall_{mali} = 0,9375.

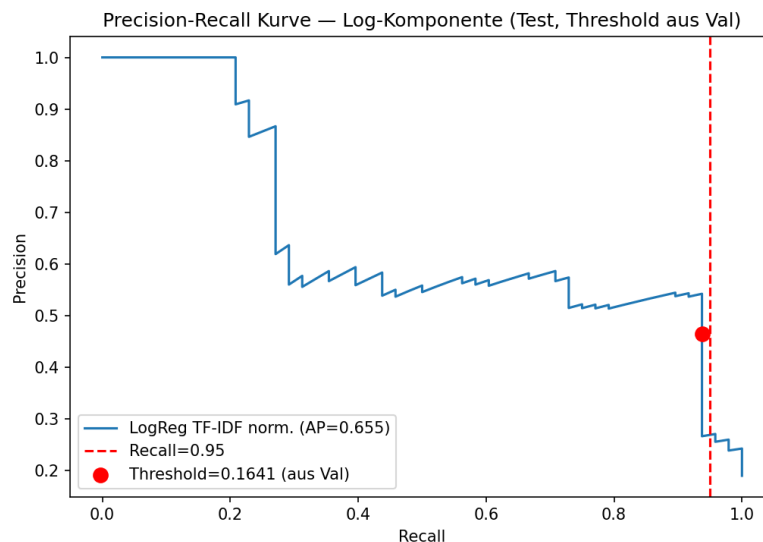


Abbildung 5: Precision-Recall-Kurve der Log-Komponente

5.6 Threshold-Sensitivity-Analyse

Zur methodischen Absicherung der Threshold-Wahl wurde eine Sensitivity-Analyse für die Log-Komponente durchgeführt. Es wurden vier Validierungs-Recall-Ziele untersucht: 0,95, 0,975, 0,99 und 1,00. Für jedes Ziel wurde der höchste Validierungsschwellenwert bestimmt, der auf dem Validierungsset das jeweilige Recall-Ziel erfüllt. Dieser Threshold wurde anschließend unverändert auf das Testset angewendet. Die resultierenden Testmetriken dienen ausschließlich der Auswertung und wurden zu keinem Zeitpunkt für die Threshold-Auswahl herangezogen. Eine testbasierte Nachoptimierung wurde bewusst nicht vorgenommen. Tabelle 9 fasst die untersuchten Validierungs-Recall-Ziele und die zugehörigen Testmetriken zusammen.

Tabelle 9: Threshold-Sensitivity der Log-Komponente

Val-Recall-Ziel	Threshold	Val-Recall	Test-Recall	Test-FPR	FPR _{anom}	FPR _{norm}	Precision	F1
0,950	0,1641	0,9623	0,9375	0,2537	0,2368	0,2636	0,4639	0,6207
0,975	0,1228	1,0000	0,9375	0,3512	0,4079	0,3178	0,3846	0,5455
0,990	0,1228	1,0000	0,9375	0,3512	0,4079	0,3178	0,3846	0,5455
1,000	0,1228	1,0000	0,9375	0,3512	0,4079	0,3178	0,3846	0,5455

Das zentrale Ergebnis ist eindeutig: Bei allen vier Recall-Zielen bleibt der Test-Recall konstant bei 0,9375. Höhere Validierungsziele führen zu einem niedrigeren Threshold und einer höheren FPR_{total} (von 0,2537 auf 0,3512), ohne den Test-Recall zu verbessern. Bei dem auf Validation ausgewählten Threshold bleiben die drei False-Negative-Cases unterhalb der Entscheidungsgrenze. Eine niedrigere Schwelle könnte den Recall erhöhen, würde jedoch voraussichtlich zusätzliche False Positives erzeugen. Dieser Trade-off wird in der Threshold-Sensitivity sichtbar. Diese Limitation wird in Abschnitt 7.4 ausführlich diskutiert.

6 Hybride Fusion und Evaluation

6.1 Fusion-Eingaben auf Case-Ebene

Beide Komponenten liefern direkt Case-Level-Scores. Die Metrik-Komponente (Abschnitt 4.3) gibt pro Case einen Score $\hat{p}_{\text{metric}}$ aus, der mit dem Validierungsschwellenwert von 0,0258 in einen binären `metric_trigger` umgewandelt wird. Die Log-Komponente (Abschnitt 5) gibt pro Case einen Log-Score $\hat{p}_{\text{log}}$ aus, der mit dem Validierungsschwellenwert von 0,164 in einen binären `log_trigger` umgewandelt wird. Ein Row-Level-Aggregationsschritt ist nicht erforderlich, da das Metrik-Modell direkt auf Case-Level-Aggregaten trainiert wurde.

Im Testset von 253 Cases ergibt die metrische Klassifikation folgende Verteilung: Für `metric_trigger=1` ergeben sich 253 Cases (100 %), während für `metric_trigger=0` kein Case vorliegt.

Dieser Befund ist zentral für die Interpretation der nachfolgenden Fusion und wird in Abschnitt 6.5 ausführlich diskutiert.

6.2 AND-Gate-Fusion und Soft-Score

Die Fusionsstufe kombiniert den binären Metriktrigger mit der log-basierten Klassifikation zu einem Alert-Signal. Es wurden zwei Fusionsstrategien untersucht:

AND-Gate-Fusion (hart):

$$\text{alert} = \mathbf{1}[\text{metric_trigger} = 1 \wedge \text{log_trigger} = 1]$$

Das AND-Gate setzt einen Alert nur dann, wenn *beide* Komponenten positiv entscheiden. Die konzeptuelle Motivation besteht darin, dass die Log-Komponente als Nachfilter wirkt: Nur Cases, die auch metrisch auffällig sind, werden auf Basis ihres Log-Kontexts abschließend bewertet [10].

Soft-Fusion:

$$s_{\text{fusion}} = \hat{p}_{\text{metric}} \times \hat{p}_{\text{log}}$$

Der Soft-Score ist das Produkt des metrischen Scores und des log-basierten Scores (vgl. [10]). Da beide Faktoren im Intervall $[0, 1]$ liegen, gilt dies auch für s_{fusion} . Auf Basis dieses Scores kann eine vollständige PR-Kurve berechnet werden.

6.3 Ablationsstudie

Tabelle 10 stellt alle vier Systemvarianten auf Case-Ebene gegenüber (Testset: 253 Cases, davon 48 positiv und 205 negativ).

Tabelle 10: Ablationsergebnisse auf Case-Ebene

Variante	Recall _{mali}	Precision	F1	FPR _{total}	FPR _{anom}	FPR _{norm}	AP	TP/FP/TN/FN
Metrics-only	1,0000	0,1897	0,3189	1,0000	1,0000	1,0000	0,2652	48/205/0/0
Log-only	0,9375	0,4639	0,6207	0,2537	0,2368	0,2636	0,6548	45/52/153/3
AND-Fusion	0,9375	0,4639	0,6207	0,2537	0,2368	0,2636	–	45/52/153/3
Soft-Fusion	0,9375	0,4545	0,6122	0,2634	0,2500	0,2713	0,5553	45/54/151/3

Alle Varianten auf Case-Ebene, $n = 253$. AP für AND-Fusion nicht anwendbar (binärer Trigger).

Die zentralen Befunde lassen sich wie folgt zusammenfassen: Metrics-only erkennt alle positiven Cases, klassifiziert aber alle negativen Cases als positiv ($\text{FPR}_{\text{total}} = 1,0000$). Log-only ist die stärkste Variante mit $\text{FPR}_{\text{total}} = 0,2537$ bei $\text{Recall}_{\text{mali}} = 0,9375$ ($\text{AP} = 0,6548$). AND-Fusion degeneriert zu Log-only, da der metrische Trigger bei allen 253 Test-Cases aktiv ist (vgl. Abschnitt 6.5). Soft-Fusion weist gegenüber Log-only denselben Recall auf, erzeugt jedoch zwei zusätzliche False Positives (54 vs. 52) und verschlechtert sich in Precision, F1, FPR und AP (F1: 0,6122 vs. 0,6207, FPR: 0,2634 vs. 0,2537, AP: 0,5553 vs. 0,6548). Soft-Fusion liefert daher keinen Zusatznutzen gegenüber Log-only. Auf diesem Benchmark liegt der wesentliche Beitrag zur False-Positive-Reduktion beim Log-Kontext. Die untersuchten Fusionsregeln liefern gegenüber Log-only keinen Mehrwert.

Abbildung 6 überlagert die PR-Kurven der Varianten mit kontinuierlichem Score. Die Metrik-only-Kurve basiert auf dem Case-Level-Score des XGBoost-Modells, die Soft-Fusion-Kurve auf dem Produkt-Score $s = \hat{p}_{\text{metric}} \times \hat{p}_{\text{log}}$. Die AND-Fusion besitzt keinen kontinuierlichen Score und erscheint daher nicht als Kurve. Alle AP-Werte beziehen sich auf dieselbe Granularität (Case-Ebene, $n = 253$).

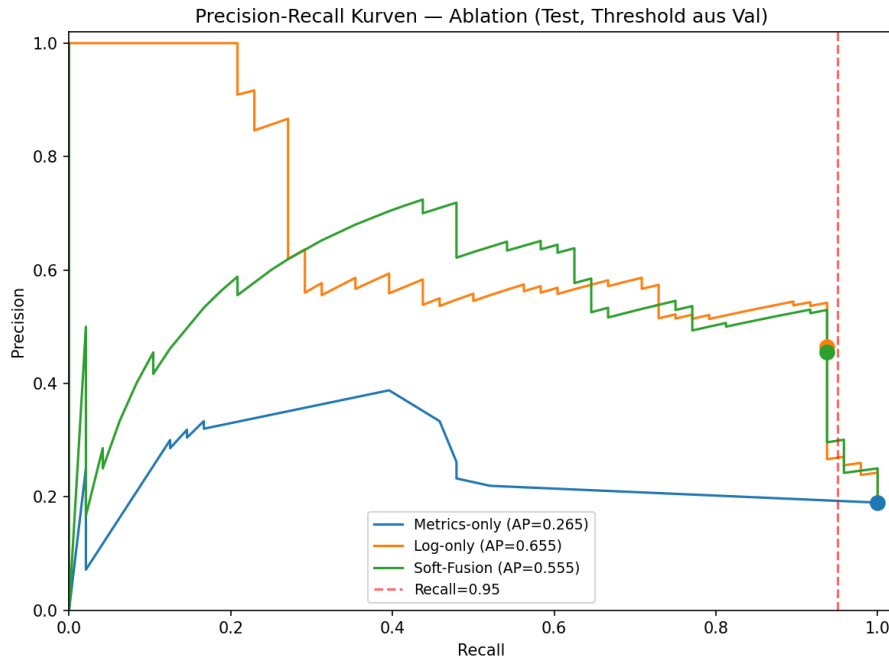


Abbildung 6: PR-Kurven der Case-Level-Ablation

Abbildung 7 zeigt den FPR-Vergleich als Balkendiagramm zwischen Metrik-Baseline, Log-Komponente und den beiden Fusionsvarianten auf Case-Ebene ($n = 253$).

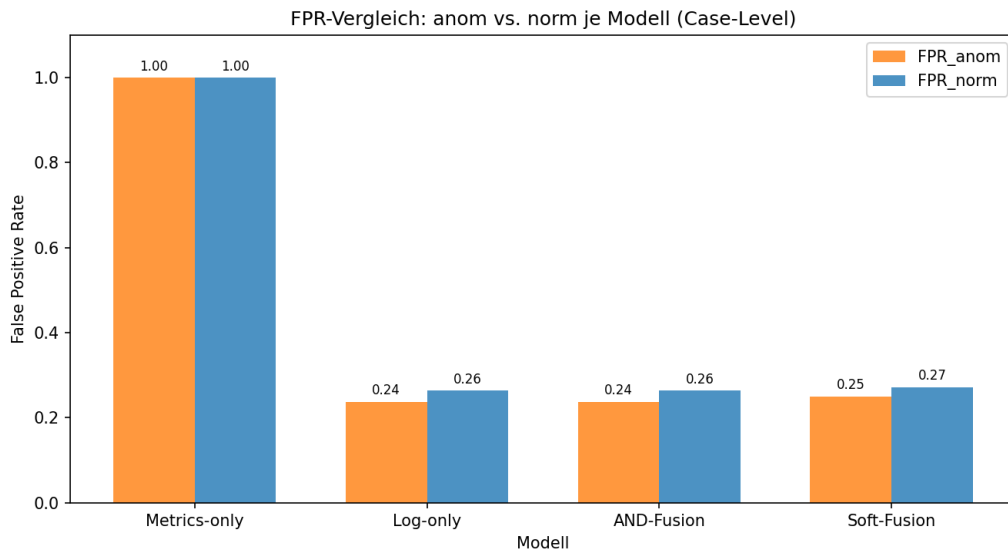


Abbildung 7: Vergleich von FPR_{anom} und FPR_{norm} zwischen den Varianten

6.4 Fehleranalyse auf Case-Ebene

Die folgenden Abschnitte analysieren qualitativ, welche Test-Cases von der Log-only-Komponente fehlerklassifiziert werden. Diese Analyse stützt sich ausschließlich auf die bereits vorliegenden Test-Scores und -Trigger aus `docs/fusion_cases.csv`. Es wurden keine Modelle neu trainiert, keine Thresholds verändert und keine testbasierte Optimierung durchgeführt.

6.4.1 False-Negative-Cases

Log-only verfehlt 3 von 48 positiven Test-Cases ($\text{Recall}_{\text{mali}} = 0,9375$). Alle drei False Negatives entstammen derselben Gruppe (`mali/scenario_6`, Szenarioname: *Rootkit / Backdoor*) und liegen mit ihren Log-Scores moderat bis deutlich unterhalb des Validierungsthresholds $\tau_{\text{log}} = 0,1641$. Tabelle 11 listet diese drei Cases auf Case-Ebene im Testset mit ihrem Log-Score, ihrem Abstand zum Threshold und einer qualitativen Beobachtung auf.

Tabelle 11: False-Negative-Cases der Log-only-Komponente

Case	Log-Score	Abstand zu τ	Qualitative Beobachtung
<code>mali_6_10</code>	0,0719	-0,0922	Moderat unter Threshold, Treffer bei cron, sshd und root erkannt
<code>mali_6_16</code>	0,0671	-0,0970	Moderat unter Threshold, Treffer bei cron und sshd erkannt
<code>mali_6_24</code>	0,0553	-0,1088	Deutlich unter Threshold, Meldungen von cron und aide erkannt

Die Logs der drei Cases enthalten Treffer aus der vordefinierten Security-Keyword-Liste (`cron`, `sshd`, `root`, `/etc/`). Da Begriffe wie `cron` oder `sshd` in regulärem Systembetrieb generisch auftreten können, reichen diese Treffer im gewählten TF-IDF-Modell mit logistischer Regression nicht für eine positive Klassifikationsentscheidung aus. Die Fehleranalyse zeigt, dass `scenario_6` (*Rootkit / Backdoor*) ein Angriffsmuster enthält, dessen Log-Signatur vom Modell bei diesem Operating-Point nicht ausreichend erfasst wird.

Tabelle 12 zeigt die Recall-Aufschlüsselung der Log-only-Komponente nach malicious Test-Szenarien. `scenario_6` erreicht einen Recall von 0,8750 (21 von 24 Cases erkannt), während `scenario_7` vollständig detektiert wird ($\text{Recall} = 1,0000$). Das Szenario 6 ist damit insgesamt schwieriger, jedoch werden 21 von 24 Cases korrekt erkannt. Nur einzelne Cases liegen unterhalb der Entscheidungsgrenze, was auf eine uneinheitliche Log-Signatur innerhalb des Szenarios hindeutet.

Tabelle 12: Malicious Test-Szenarien: Recall

Szenario	Name	n	TP	FN	Recall
<code>mali/scenario_6</code>	Rootkit / Backdoor	24	21	3	0,8750
<code>mali/scenario_7</code>	Brute-force / Credential Stuffing Attack	24	24	0	1,0000
Gesamt		48	45	3	0,9375

Bei dem auf Validation ausgewählten Threshold bleiben die drei False-Negative-Cases unterhalb der Entscheidungsgrenze. Eine niedrigere Schwelle könnte den Recall erhöhen, würde jedoch voraussichtlich zusätzliche False Positives erzeugen. Dieser Trade-off wird in der Threshold-Sensitivity sichtbar (Abschnitt 5.6). Eine nachträgliche Anpassung auf Basis der Testmenge wäre jedoch methodisch nicht zulässig.

6.4.2 False-Positive-Cases nach Szenario

Die 52 False Positives von Log-only ($FPR_{\text{total}} = 0,2537$) sind nicht gleichmäßig auf Szenarien verteilt. Tabelle 13 schlüsselt die Fehlklassifikationen für alle anom- und norm-Szenarien im Testset auf (205 negative Cases).

Tabelle 13: False-Positive-Analyse nach Szenario für Log-only

Typ	Szenario	Cases	FP	FPR	$\tilde{s}_{\log}$
anom	scenario_17	18	11	0,6111	0,3218
anom	scenario_5	30	6	0,2000	0,1405
anom	scenario_13	28	1	0,0357	0,0937
norm	scenario_8	30	30	1,0000	0,5226
norm	scenario_15	30	2	0,0667	0,0861
norm	scenario_6	24	1	0,0417	0,0717
norm	scenario_4	45	1	0,0222	0,0534

FPR und $\tilde{s}_{\log}$ beziehen sich ausschließlich auf Log-only.

Die Fehleranalyse zeigt, dass die Fehlklassifikationen szenarioabhängig konzentriert sind.

Abbildung 8 stellt die szenariospezifische Verteilung der False Positives der Log-only-Komponente grafisch dar, aufgeschlüsselt nach anom (links) und norm (rechts). Die Annotation zeigt die szenariospezifische FPR, rote Balken markieren Szenarien mit $FPR = 1,0$.

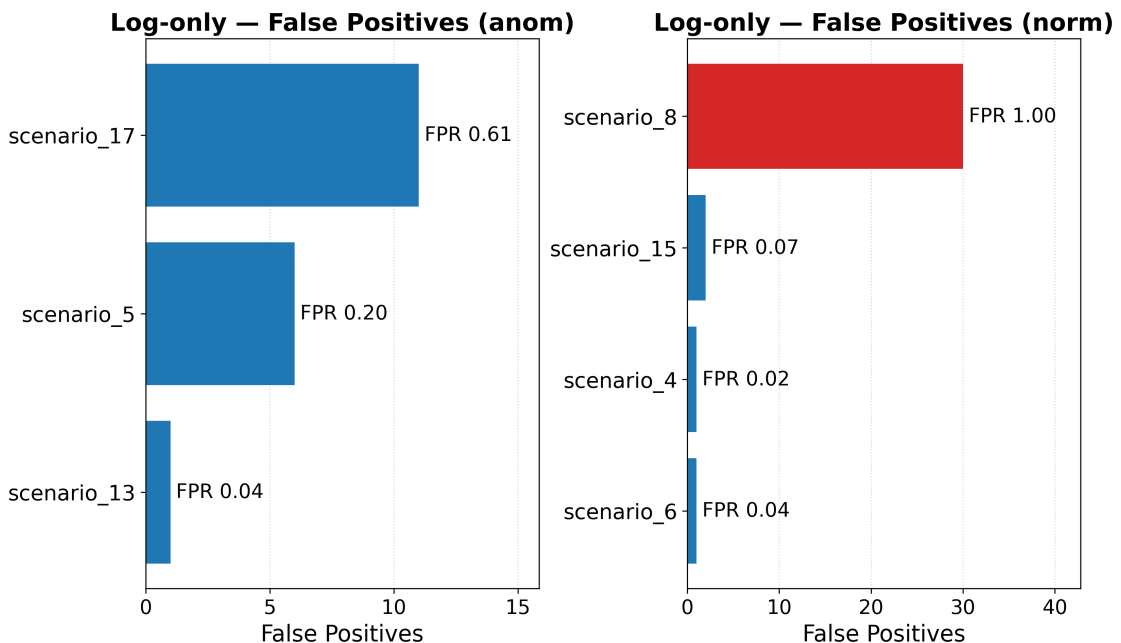


Abbildung 8: False Positives nach Szenario für Log-only

Im anom-Bereich dominiert `anom/scenario_17` mit einer FPR von 0,6111 (11 von 18 Cases). Alle weiteren anom-Szenarien weisen deutlich niedrigere Raten auf. Für `anom/scenario_17` liegt kein offiziell bekannter Szenarioname vor. Die Log-Scores dieses Szenarios liegen im Schnitt bei 0,3218 und damit deutlich oberhalb des verwendeten Log-Thresholds von $\tau_{\log} = 0,164056$, was auf systematisch erhöhte Treffer aus der vordefinierten Security-Key-Liste hinweist, die das Modell stark mit mali-Klassen assoziiert.

Im norm-Bereich fällt `norm/scenario_8` (*Search Engine Aggressive Crawl*) besonders auf: Alle 30 norm-Cases des Szenarios werden positiv klassifiziert ($FPR = 1,0000$, $\bar{s}_{\log} = 0,5226$). Ein aggressiver Web-Crawler erzeugt in Server-Logs oberflächlich attackenähnliche Muster, etwa dichte HTTP-Request-Serien, ungewöhnliche User-Agent-Strings oder wiederholte Zugriffsversuche, obwohl es sich um ein normales, wenn auch ressourcenintensives Ereignis handelt. Das TF-IDF-Modell kann diesen Unterschied allein anhand der Termstatistiken nicht zuverlässig auflösen. Dies erklärt, warum $FPR_{\text{norm}} = 0,2636$ insgesamt etwas höher liegt als $FPR_{\text{anom}} = 0,2368$. Eine kausale Erklärung der szenariospezifischen Log-Muster erfordert eine tiefere manuelle Log-Inspektion und liegt außerhalb des Kernumfangs dieser Arbeit.

6.5 AND-Gate-Degeneration

Ein wesentlicher empirischer Befund dieser Arbeit ist die vollständige Degeneration des AND-Gates. Alle 253 Test-Cases erhalten ein `metric_trigger=1` ($253/253 = 100\%$). Jeder Case löst den Metrikdetektor aus. Die AND-Fusion ist damit funktional identisch zur reinen Log-Filterung: Da die Bedingung `metric_trigger=1` für jeden Case erfüllt ist, gilt $AND = \text{metric_trigger} \wedge \text{log_trigger} = \text{log_trigger}$. Durch die Einbeziehung des Log-Kontexts sinkt die FPR gegenüber Metrics-only. Da der metrische Trigger jedoch alle Testfälle aktiviert, erzielt die AND-Fusion auf dem Testset keinen zusätzlichen Fusionsgewinn gegenüber Log-only. Die Konfusionsmatrizen von AND-Fusion und Log-only sind exakt gleich.

Die Ursache liegt in der Kalibrierung des Metrik-Detektors. Der niedrige Validierungsthreshold von 0,0258 markiert alle Test-Cases als metrisch auffällig, unabhängig von ihrer tatsächlichen Klasse. Die 25 Case-Level-Features der fünf universellen Metriken liefern auf diesem Benchmark keine ausreichende Trennkraft, um anom- und norm-Cases von mali-Cases zu unterscheiden. Viele technische Anomalien und Normalszenarien weisen ähnlich hohe Metrik-Aggregate wie Malware-Aktivitäten auf. Das AND-Gate, das konzeptionell als Zweistufenfilter gedacht ist, verliert dadurch seine Filterfunktion auf Metrikseite vollständig.

Diese Degeneration ist kein Implementierungsfehler, sondern ein direktes Ergebnis der Nicht-Selektivität des Metrik-Detektors auf Case-Ebene in diesem Benchmark. Sie muss bei der Interpretation der Ergebnisse transparent benannt werden. Eine Möglichkeit, die Degeneration zu überwinden, wäre der Einsatz eines ausdrucksstärkeren metrischen Detektors (z. B. mit typspezifischen Features oder anderen Modellklassen). Dies liegt jedoch außerhalb des Rahmens dieser Arbeit.

6.6 Qualitätssicherung und Reproduzierbarkeit

Die Qualitätssicherung umfasst 67 automatisierte Tests in 14 Testklassen, die zentrale Implementierungs- und Konsistenzbedingungen absichern (`test_pipeline.py` im Verzeichnis `tests/`, `pytest`-Framework):

1. **Canonical Matrix / Preprocessing** (`TestBuildCanonicalMatrix`, 4 Tests): Prüft die korrekte Behandlung fehlender Spalten, die Erhaltung von NaN vor dem Split, die vollständige NaN-Bereinigung nach evaluationsrelevanter Imputation sowie die Konvertierung von in Anführungszeichen gespeicherten numerischen Zeichenketten in numerische Werte.
2. **Zeilen-Imputation** (`TestRowLevelImputation`, 6 Tests): Verifiziert, dass keine Medianimputation vor dem Split erfolgt, Imputationsmediane ausschließlich aus Trainingszeilen stammen, derselbe Median je Feature auf alle `dataset_type`-Werte und Splits angewendet wird und die finalen 25 Case-Level-Features keine NaN enthalten.
3. **Fusion / Soft-Score** (`TestFusionLogic`, 3 Tests): Verifiziert die AND-Gate-Wahrheitstabelle auf allen vier Eingangskombinationen sowie die Wertebereichseigenschaft des Soft-Scores ($s_{\text{fusion}} \in [0, 1]$).
4. **Datenintegrität / GroupShuffleSplit** (`TestDataIntegrity`, 2 Tests): Stellt sicher, dass keine Gruppe in mehr als einem der drei Splits gleichzeitig vorkommt, und prüft das Vorhandensein aller fünf universellen Features in sämtlichen Datensatztypen.
5. **Algorithmische Logik** (`TestAlgorithmicLogic`, 4 Tests): Prüft die Threshold-Auswahllogik, die Recall-Nebenbedingung nach Threshold-Anwendung sowie die Abwesenheit testexklusiver Terme im TF-IDF-Vokabular.
6. **Validierungs-Split** (`TestValidationSplit`, 6 Tests): Verifiziert die Disjunktheit aller drei Splits nach Gruppenkennung sowie die ausschließliche Verwendung von Trainingsdaten für Imputations-Mediane.
7. **Threshold-Sensitivity** (`TestThresholdSensitivity`, 5 Tests): Prüft, dass die Sensitivity-Tabelle alle vier Recall-Ziele enthält, dass Thresholds ausschließlich aus Validierungsdaten stammen und dass Metriken im Einheitsintervall liegen.
8. **Case-Level Metrik-Baseline** (`TestCaseLevelMetricBaseline`, 6 Tests): Verifiziert die Gruppendisjunktheit auf Case-Ebene, eine Zeile pro Case in den Feature-Artefakten, die Abwesenheit von Row-Level-Leakage in den Features sowie die validierungsbasierte Threshold-Wahl.
9. **Log-Normalisierung und Scores** (`TestLogNormalizationAndScores`, 6 Tests): Prüft, dass die Normalisierungsfunktion IPs, Zeitstempel, PIDs und isolierte Zahlen ersetzt, sicherheitsrelevante Terme der vordefinierten Keyword-Liste erhält, Case-Level-Scores korrekt vorliegen und das TF-IDF-Vokabular ausschließlich auf Trainingsdaten gefittet wird.

10. **Case-Level Fusion** (TestCaseLevelFusion, 7 Tests): Verifiziert, dass die Fusion ausschließlich Case-Level-Scores verwendet, eine Zeile pro Test-Case vorliegt, der Soft-Fusion-Threshold aus Validation stammt und die FPR-Aufschlüsselung nach Datensatztyp konsistent ist.
11. **False-Negative-Analyse** (TestFalseNegativeCases, 3 Tests): Prüft, dass genau 3 FN-Cases vorliegen, alle dem Typ mali entstammen und ihre Log-Scores unterhalb des Validierungsthresholds liegen.
12. **False-Positive-Szenario-Analyse** (TestFalsePositiveByScenario, 5 Tests): Verifiziert die szenariospezifische FP-Aggregation, die Konsistenz der FPR-Berechnung und die korrekte Identifikation der Top-Szenarien (norm/scenario_8, anom/scenario_17).
13. **Fehleranalyse ändert keine Hauptergebnisse** (TestErrorAnalysisDoesNotChangeFinalResults, 1 Test): Stellt sicher, dass die diagnostische Fehleranalyse keine Modelle neu trainiert, keine Thresholds verändert und keine finalen Hauptmetriken beeinflusst.
14. **Fresh-run-Determinismus** (TestFreshRunSplitDeterminism, 9 Tests): Verifiziert, dass die Split-Erzeugung keine bestehende split_assignments.csv voraussetzt, gleiche Seeds bit-identische Zuordnungen liefern, alle drei Splits auf Case- und Gruppenebene disjunkt sind, Zeilenmediane ausschließlich aus Trainingszeilen stammen und der kanonische Split deterministisch reproduzierbar ist.

Die Reproduzierbarkeit wird durch fixierte Zufallsseeds, gespeicherte Split-Zuordnungen, eine dokumentierte Ausführungsreihenfolge, eine spezifizierte Umgebung, versionierte Ergebnisartefakte und den finalen Repository-Commit unterstützt: Der finale Testsplit verwendet random_state=42. Der interne Validierungsanteil wird ohne Zugriff auf das Testset über Seed 0 aus einer vordefinierten Kandidatenliste gewählt, sodass ausreichend positive Validation-Cases enthalten sind. Das XGBoost-Training verwendet ebenfalls random_state=42. Die Datei docs/fusion_cases.csv enthält alle 253 Test-Cases mit ihren Zwischen-Scores (metric_score, log_score, metric_trigger, log_trigger, and_trigger, score_fusion und soft_fusion_trigger) als nachvollziehbarer Audit-Trail.

7 Diskussion und Limitationen

7.1 Beantwortung der Forschungsfrage

Die Forschungsfrage lautete, ob eine zweistufige hybride Pipeline die False-Positive-Rate auf CloudAnoBench deutlich senken kann, ohne die Recall-Rate unter 0,95 zu drücken. Die Ablationsergebnisse in Abschnitt 6.3 beantworten diese Frage differenziert: Die Log-Komponente reduziert die FPR_{total} von 1,0000 (Metrics-only) auf 0,2537. Der Log-Kontext ist damit der bestimmende Faktor der False-Positive-Reduktion. Der Test-Recall_{mali} beträgt 0,9375, was die angestrebte Schwelle von 0,95 knapp verfehlt (3 von 48 positiven Cases nicht detektiert). Die Fusionsstufe liefert keinen Zusatznutzen gegenüber Log-only.

Das entscheidende Systemelement ist die Log-Komponente. Die log-basierte Klassifikation auf Case-Ebene identifiziert anhand textueller Systemereignismuster, welche Cases nicht als malicious einzustufen sind. Die Fusionsstufe trägt in dieser Konfiguration keinen eigenständigen Beitrag: AND-Fusion degeneriert zu Log-only ($\text{metric_trigger} = 253/253 = 100\%$), und Soft-Fusion verschlechtert die Diskriminanz durch Multiplikation mit dem schwachen Metrik-Score.

7.2 Rolle des Log-Kontexts

Die deutliche FPR-Reduktion durch die Log-Komponente lässt sich durch den zusätzlichen Informationsgehalt der Log-Daten erklären. Log-Meldungen enthalten semantisch reichhaltige Ereignisbeschreibungen, die Ursache und Art eines Betriebsvorgangs qualitativ charakterisieren können. Metriken hingegen messen quantitative Systemzustände und können zwischen unterschiedlichen Ereignistypen mit identischer Intensität nicht unterscheiden. TF-IDF kann diskriminierende Terme identifizieren, die in malignen Cases systematisch häufiger oder seltener auftreten als in normalen oder anomalen Cases.

Die Analyse auf Case-Ebene statt Zeilenebene ist dabei konzeptuell angemessen: Die Entscheidung, ob ein Security-Alert ausgelöst werden soll, bezieht sich auf eine Beobachtungseinheit (Case) als Ganzes, nicht auf einen einzelnen Messpunkt.

7.3 Grenzen der Metrik-Komponente

Der metrische Detektor erzielt mit einer AP von 0,2652 eine schwache Diskriminationsleistung auf Case-Ebene. Die Schema-Leakage-Vermeidung (vgl. Abschnitt 3.3) beschränkt das Modell auf 25 Case-Level-Features der fünf universellen Metriken (je Metrik: Mittelwert, Maximum, Standardabweichung, 95. Perzentil, Trend). Diese Aggregate bieten keine ausreichende Trennkraft zwischen mali-, anom- und norm-Cases: Viele technische Anomalien und Normalszenarien erzeugen ähnlich hohe Metrik-Spitzen wie Malware-Aktivitäten. Die Konsequenz ist ein niedriger operativer Threshold von 0,0258, der alle 253 Test-Cases als metrisch auffällig markiert ($\text{FPR}_{\text{total}} = 1,0000$, $\text{TN} = 0$) und die vollständige Degeneration des AND-Gates bedingt.

Ein metrischer Detektor mit höherer Ausdrucksstärke, etwa durch typspezifische Features oder tiefere Modellklassen, könnte diese Einschränkungen möglicherweise mildern, ist jedoch mit dem Risiko erhöhten Schema-Leakages verbunden.

7.4 Methodische Limitationen

Mehrere methodische Einschränkungen begrenzen die Generalisierbarkeit der Befunde und müssen transparent benannt werden.

Einzelner Train-/Validierungs-/Test-Split. Es wurde lediglich ein einziger gruppenbasierter Train-/Validierungs-/Test-Split durchgeführt. Die Streuung der Ergebnismetriken über verschiedene Splits ist damit unbekannt. Threshold-Wahl und Early Stopping erfolgen ausschließlich auf dem Validierungsset, sodass das Testset als unberührter Holdout dient. Da die Evaluation auf einem einzigen festen

Split basiert, lässt sich die Stabilität der Ergebnisse gegenüber anderen Gruppenzuordnungen nicht beurteilen. Bootstrap-Konfidenzintervalle oder wiederholte Splits könnten diese Limitation in zukünftigen Arbeiten adressieren.

Kleine positive Testmenge. Das Testset enthält lediglich 48 positive Cases (mali). Über das im Ausblick berichtete Wilson-Konfidenzintervall für den Recall hinaus wurden keine Konfidenzintervalle für die übrigen Metriken berechnet. Die Punktschätzungen der FPR und des Recalls sind daher mit entsprechender Vorsicht zu interpretieren.

Begrenzte szenariospezifische Testabdeckung der mali-Klasse. Der Testsplit enthält lediglich zwei malicious Szenarien (mali/scenario_6 und mali/scenario_7). Der berichtete malicious Recall von 45/48 ist daher ein Befund auf diesem festen, scenario-aware Holdout-Split und darf nicht als empirische Abdeckung aller zwölf malicious Szenarien im Datensatz interpretiert werden. Der scenario-aware Split bleibt methodisch korrekt. Die Einschränkung betrifft allein die externe Breite der Recall-Aussage.

Synthetischer Datensatz. CloudAnoBench ist ein synthetisch generierter Datensatz und kein aufgezeichneter Produktions-Traffic. Ob die gelernten Textmuster auf andere Cloud-Umgebungen, andere Log-Formate oder reale Angriffe übertragbar sind, wurde im Rahmen dieser Arbeit nicht untersucht, da die Evaluation ausschließlich auf CloudAnoBench basiert. Arp et al. [2] weisen allgemein darauf hin, dass die Übertragbarkeit von im Labor evaluierten ML-Sicherheitsmodellen auf reale Einsatzumgebungen systematisch überschätzt werden kann.

Einheitliche Case-Level-Evaluation. In der finalen Pipeline werden Metrik-Baseline, Log-Komponente und Fusion auf derselben Granularität evaluiert: eine Zeile pro Case. Dadurch sind Recall, Precision, F1, AP sowie FPR_{total} , FPR_{anom} und FPR_{norm} direkt vergleichbar. Die Aussagekraft bleibt dennoch durch den einzelnen Holdout-Split begrenzt. Insbesondere beruhen die Recall-Werte im Testset auf 48 positiven mali-Cases.

Recall-Gap der Log-Komponente. Der Test-Recall der Log-Komponente verfehlt die angestrebte 0,95-Schwelle knapp. Die Threshold-Sensitivity-Analyse (vgl. Abschnitt 5.6) zeigt, dass keines der getesteten validierungsbasierten Recall-Ziele diesen Gap auf dem Testset schließt: Die betroffenen FN-Cases liegen unterhalb des niedrigsten Validierungsthresholds, der auf dem Validierungsset vollständigen Recall erzielt. Eine nachträgliche testbasierte Threshold-Optimierung wurde bewusst nicht durchgeführt, da sie dem Prinzip der leakage-sicheren Evaluation widerspräche [3].

Soft-Fusion ohne Mehrwert gegenüber Log-only. Die Soft-Fusion-Strategie ($s = \hat{p}_{metric} \times \hat{p}_{log}$) erzielt eine AP von 0,5553, die unter der AP der Log-Komponente allein (0,6548) liegt. F1 und FPR sind ebenfalls schlechter (F1: 0,6122 vs. 0,6207, FPR_{total} : 0,2634 vs. 0,2537). Die Ursache liegt darin, dass der metrische Score bei allen Test-Cases nahezu konstant hoch ist: Die Multiplikation nivelliert die Diskriminanz des Log-Scores, ohne einen eigenständigen Informationsgewinn zu liefern. Tax et al. [15] zeigen, dass die Produktregel bei abhängigen oder

redundanten Repräsentationen schlechter abschneiden kann. Eine alternative Fusionsgewichtung könnte diesen Nachteil möglicherweise beheben, lag jedoch außerhalb des festgelegten Untersuchungsrahmens.

Prototypischer Implementierungscharakter. Die gesamte Pipeline ist in Jupyter Notebooks implementiert und als Forschungsprototyp zu verstehen. Es existiert keine paketierte, produktionsreife Software-Implementierung.

Keine tiefe manuelle Log-Inspektion einzelner norm-Cases. Die Evaluation weist die FPR für norm-Cases (planmäßiger Betrieb, darunter Backup-Vorgänge und Batch-Jobs) separat aus. Für die beste Variante (Log-only) beträgt $FPR_{\text{norm}} = 0,2636$ (34 von 129 Cases). Eine tiefe manuelle Log-Inspektion einzelner falsch positiv klassifizierter norm-Cases lag außerhalb des festgelegten Untersuchungsrahmens dieser Arbeit. Die Zwischenergebnisse aller 253 Test-Cases, einschließlich metrischer und log-basierter Scores, sind in `docs/fusion_cases.csv` dokumentiert und stehen für Anschlussanalysen bereit.

7.5 Skalierbarkeit als konzeptuelle Einordnung

Konzeptuell ließe sich die Pipeline als verteilbare Abfolge von Filter- und Verifikationsschritten auffassen, etwa in Form einer Map/Reduce-artigen Aufteilung: Die metrische Vorab-Filterung (Phase 2) lässt sich als parallele Map-Operation auf Case-Ebene interpretieren, die log-basierte Verifikation (Phase 3) als fallweise Klassifikation auf dem normalisierten Log-Text.

Es ist jedoch ausdrücklich festzuhalten, dass diese Skalierbarkeitsbetrachtung konzeptueller Natur ist. Eine verteilte Implementierung wurde im Rahmen dieser Arbeit weder realisiert noch gemessen. Die gemessenen Laufzeiten der einzelnen Notebook-Phasen sind als indikative Einzelmessungen zu verstehen und hängen von Ausführungskontext, Hardware und Notebook-Umgebung ab. Sie werden daher nicht als belastbarer Benchmark herangezogen. Für die Bewertung dieser Arbeit entscheidend ist allein die konzeptuelle Zerlegung der Pipeline in getrennte, grundsätzlich parallelisierbare Schritte.

Eine Einbettung der Pipeline in Data-Intelligence-Plattformen wie Databricks oder Snowflake sowie ein automatisiertes Modell-Lifecycle-Management, etwa über MLflow zur Überwachung von Model Drift in dynamischen Cloud-Umgebungen, wären naheliegende Erweiterungen für eine produktionsnahe Umsetzung. Diese Aspekte wurden in der vorliegenden Arbeit weder implementiert noch empirisch evaluiert und liegen außerhalb des Rahmens dieser Benchmark-Studie. Die Skalierbarkeitsbetrachtung bleibt daher auf die konzeptuelle Zerlegung der Pipeline in verteilbare Filter- und Verifikationsschritte beschränkt.

8 Fazit und Ausblick

8.1 Zusammenfassung

Diese Arbeit hat auf dem synthetischen Benchmark CloudAnoBench untersucht, inwiefern Log-Kontext die False-Positive-Rate einer metrikbasierten Cloud-Anomalieerkennung reduzieren kann. Dazu wurde eine reproduzierbare Case-Level-Pipeline entwickelt, die vier Varianten vergleicht: einen XGBoost-Klassifikator auf 25 aggregierten Case-Level-Systemmetriken (Metrics-only), eine log-basierte Klassifikation mit TF-IDF und logistischer Regression auf normalisierten Log-Texten (Log-only) sowie zwei Fusionsvarianten (AND-Fusion, Soft-Fusion). Der Aufbau folgt einem scenario-aware dreistufigen Split (Train / Validation / Test) ohne Gruppenüberlappung. Threshold-Wahl und Early Stopping wurden ausschließlich auf dem Validierungssset vorgenommen, das Testset diente nur der finalen Evaluation. Die Qualitätssicherung umfasst 67 automatisierte Tests in 14 Testklassen, die zentrale Implementierungs- und Konsistenzbedingungen absichern. Die Reproduzierbarkeit wird zusätzlich durch fixierte Zufallsseeds (Testsplit: `random_state=42`), gespeicherte Split-Zuordnungen und die dokumentierte Ausführungsreihenfolge unterstützt.

Kapitel 2 legte die konzeptionellen und methodischen Grundlagen, darunter Cloud-Anomalieerkennung, unbalancierte Klassifikation, XGBoost, TF-IDF und leakage-sichere Validierung. Kapitel 3 beschrieb den Datensatz, die Vorverarbeitung und den gruppenbasierten Split. Die Kapitel 4 bis 6 untersuchten die metrische Baseline, die log-basierte Verifikationskomponente sowie die Fusionsstrategien und werteten die Ergebnisse auf Case-Ebene aus. Kapitel 7 diskutierte die Befunde im Hinblick auf die Forschungsfrage und benannte die methodischen Limitationen.

8.2 Zentrale Ergebnisse

Die Ablation auf dem Testset (253 Cases, davon 48 mali, 76 anom, 129 norm) ergibt folgendes Bild:

Metrics-only. Erkennt alle 48 positiven Cases ($\text{Recall}_{\text{mali}} = 1,0000$), klassifiziert aber gleichzeitig alle 205 negativen Cases als positiv ($\text{FPR}_{\text{total}} = 1,0000$, $\text{TN} = 0$). Die 25 Case-Level-Aggregate der fünf universellen Metriken liefern auf diesem Benchmark keine ausreichende operative Selektivität am gewählten High-Recall-Operating-Point.

Log-only. Ist die stärkste Variante: $\text{FPR}_{\text{total}} = 0,2537$ ($\text{FPR}_{\text{anom}} = 0,2368$, $\text{FPR}_{\text{norm}} = 0,2636$) bei $\text{Recall}_{\text{mali}} = 0,9375$ und $\text{AP} = 0,6548$. Damit werden 45 von 48 positiven Cases erkannt, während 3 Cases undetektiert bleiben (False Negatives).

AND-Fusion. Ist funktional identisch zu Log-only: Der metrische Trigger ist bei allen 253 Test-Cases aktiv ($\text{metric_trigger} = 253/253 = 100\%$), sodass das AND-Gate vollständig degeneriert.

Soft-Fusion. Weist gegenüber Log-only denselben Recall auf, erzeugt jedoch zwei zusätzliche False Positives (54 vs. 52) und verschlechtert sich in Precision, F1, FPR und AP: $F1 = 0,6122$ (vs. $0,6207$), $FPR_{\text{total}} = 0,2634$ (vs. $0,2537$), $AP = 0,5553$ (vs. $0,6548$). Soft-Fusion liefert daher keinen Zusatznutzen gegenüber Log-only.

8.3 Beantwortung der Forschungsfrage

Die Forschungsfrage lautete, ob eine zweistufige hybride Pipeline die False-Positive-Rate auf CloudAnoBench deutlich senken kann, ohne die Recall-Rate unter 0,95 zu drücken. Die Ergebnisse beantworten diese Frage differenziert.

Log-Kontext reduziert die FPR_{total} deutlich: Gegenüber der nicht-selektiven Metrik-Baseline ($FPR = 1,0000$) erreicht Log-only eine FPR_{total} von $0,2537$. Der Hauptnutzen liegt damit primär in der log-basierten Modellierung, nicht in der Fusion. Die ursprüngliche Erwartung, dass eine kombinierte Metrik-Log-Pipeline besser abschneidet als Log-only allein, wird durch die Ergebnisse nicht bestätigt: Weder AND-Fusion noch Soft-Fusion liefern einen Zusatznutzen.

Der Test-Recall_{mali} von $0,9375$ ($45/48$) verfehlt die angestrebte Schwelle von $0,95$ knapp. Bei dem auf Validation ausgewählten Threshold bleiben die drei False-Negative-Cases unterhalb der Entscheidungsgrenze. Eine niedrigere Schwelle könnte den Recall erhöhen, würde jedoch voraussichtlich zusätzliche False Positives erzeugen. Dieser Trade-off wird in der Threshold-Sensitivity sichtbar. Dieser Befund ist methodisch transparent: Eine testbasierte Nachoptimierung wurde bewusst nicht vorgenommen.

Die Befunde gelten für CloudAnoBench unter den gewählten Konfigurationen. Die Übertragbarkeit der Ergebnisse auf andere Benchmark-Datensätze oder reale Infrastrukturen muss offen bleiben, da die Evaluation ausschließlich auf CloudAnoBench durchgeführt wurde.

8.4 Methodische Bedeutung

Mehrere methodische Entscheidungen haben die Validität der Evaluation wesentlich bestimmt.

Case-Level-Auswertung. Die Auswertung auf Case-Ebene ist für die Security-Detection konzeptuell angemessen: Ein Alert-Entscheid bezieht sich auf eine Beobachtungseinheit als Ganzes, nicht auf einen einzelnen Messpunkt. Ein ausschließlich Row-Level-Vergleich hätte die AP-Werte strukturell verzerrt und einen direkten Vergleich zwischen Metrik- und Log-Komponente unmöglich gemacht.

Log-Normalisierung. Das Ersetzen von Zeitstempeln, IP-Adressen, PIDs, Ports und isolierten Zahlen durch generische Token vor der TF-IDF-Vektorisierung stellt sicher, dass das Modell auf semantisch relevanten Mustern und nicht auf synthetischen Artefakten des Benchmark-Generators trainiert wird. Dieser Schritt ist methodisch sauber, weil er deterministisch und label-unabhängig ist.

Getrennte FPR-Auswertung. Die getrennte Auswertung für anom und norm zeigt, dass beide Negativtypen relevant und ähnlich schwer von mali zu trennen sind (Log-only: $FPR_{anom} = 0,2368$, $FPR_{norm} = 0,2636$). Eine aggregierte FPR allein hätte diese Differenzierung verborgen.

Validierungsbasierte Threshold-Wahl. Die Threshold-Wahl auf dem Validierungsset ohne testbasierte Anpassung verhindert Informationslecks aus dem Testset und macht die Evaluation unabhängig von Post-hoc-Optimierungen, auch wenn dies den knapp verfehlten Test-Recall bedingt.

8.5 Limitationen

Mehrere Einschränkungen begrenzen die Generalisierbarkeit der Befunde.

Verfehlte Recall-Zielschwelle. Der Test-Recall_{mali} von 0,9375 liegt knapp unter der angestrebten Schwelle von 0,95. Drei positive Cases bleiben undetektiert. Die Fehleranalyse (vgl. Abschnitt 6.4) zeigt, dass alle drei False Negatives aus demselben Szenario (mali/scenario_6, *Rootkit / Backdoor*) stammen und mit Log-Scores moderat bis deutlich unter dem Validierungsthreshold liegen. Dies deutet auf ein szenariospezifisches Angriffsmuster hin, dessen Log-Signatur vom TF-IDF-Modell nicht ausreichend erfasst wird. Bei dem auf Validation ausgewählten Threshold bleiben diese Cases unterhalb der Entscheidungsgrenze. Eine niedrigere Schwelle könnte den Recall erhöhen, würde jedoch voraussichtlich zusätzliche False Positives erzeugen. Dieser Trade-off wird in der Threshold-Sensitivity (vgl. Abschnitt 5.6) sichtbar.

Szenarioabhängige Konzentration der False Positives. Die Fehleranalyse zeigt, dass die False Positives von Log-only nicht gleichmäßig auf Szenarien verteilt sind. Im anom-Bereich konzentrieren sich 11 von 18 Fehlklassifikationen auf scenario_17 ($FPR = 0,6111$). Im norm-Bereich weist scenario_8 eine FPR von 1,0000 auf. Alle 30 norm-Cases dieses Szenarios werden positiv klassifiziert. Eine kausale Erklärung erfordert eine tiefere semantische Log-Inspektion und bleibt zukünftiger Arbeit vorbehalten.

Geringe Diskriminationskraft der Metrik-Komponente. Die 25 aggregierten Case-Level-Features der fünf universellen Metriken reichen nicht aus, um mali-Cases von anom- und norm-Cases zu trennen. Typspezifische Features oder ausdrucksstärkere Modellklassen könnten diese Einschränkung möglicherweise mildern, bergen aber das Risiko von Schema-Leakage.

Einfache Fusionsstrategien. Die untersuchten Fusionsmethoden, binäres AND-Gate und Produkt-Score, gehören zu den einfachsten möglichen Kombinationsregeln. Die Ergebnisse zeigen nicht, dass Log-Fusion im Allgemeinen keinen Nutzen hat, sondern dass diese spezifischen einfachen Fusionsregeln auf diesem Benchmark keinen Zusatznutzen gegenüber Log-only liefern.

Einzelner Split. Es wurde lediglich ein einziger scenario-aware Train-/Validierungs-/Test-Split verwendet. Die Streuung der Ergebnisse über verschiedene Splits ist unbekannt.

Synthetischer Datensatz. CloudAnoBench basiert auf kontrollierten Anomalie-Injektionen. Ob die gelernten Log-Muster auf reale Infrastrukturdaten mit natürlichen Klassenverteilungen und variablen Log-Formaten übertragbar sind, wurde im Rahmen dieser Arbeit nicht untersucht, da die Evaluation ausschließlich auf CloudAnoBench basiert.

Keine Sequenzmodelle oder semantische Log-Encoder. TF-IDF und logistische Regression sind eine interpretierbare, aber begrenzte Baseline. Die Untersuchung strukturierter Log-Parser und tiefer Sprachmodelle lag außerhalb des festgelegten Rahmens dieser Arbeit.

8.6 Ausblick

Aus den Befunden dieser Arbeit ergeben sich mehrere konkrete Ansatzpunkte für zukünftige Arbeiten:

Ausdrucksstärkere Metrik-Features. Zeitfenster-Aggregate, typspezifische Features oder tiefe Sequenzmodelle auf Zeitreihenbasis könnten die Diskriminationskraft der metrischen Komponente verbessern, sofern Schema-Leakage vermieden wird. Eine selektivere Metrik-Komponente würde auch die AND-Gate-Fusion aus ihrer Degeneration befreien.

Robustere Fusionsregeln. Kalibrierte Scores, gewichtete Kombinatoren oder lernbasierte Fusionsmodelle könnten den Informationsgehalt beider Modalitäten besser ausnutzen als das ungewichtete Produkt oder das binäre AND-Gate. Ob solche Regeln auf CloudAnoBench einen Vorteil gegenüber Log-only erzielen, bleibt eine offene Frage.

Sequenzielle Log-Modelle und strukturiertes Log-Parsing. Ob strukturbewusste Log-Parser, vortrainierte Sprachmodelle oder Embedding-Methoden die FPR weiter reduzieren, wäre eine naheliegende Erweiterung der Log-Komponente über die TF-IDF-Baseline hinaus.

Weitere Splits und statistische Absicherung. Wiederholte Splits mit unterschiedlichen Zufallszuständen oder Bootstrap-Konfidenzintervalle würden die Stabilität der Ergebnisse quantifizierbar machen. Für den beobachteten Recall von 45/48 ergibt sich ein approximatives 95%-Wilson-Konfidenzintervall von $[0,832; 0,979]$. Der Zielwert 0,95 liegt damit innerhalb des Intervalls. Dieses Ergebnis ist jedoch im Kontext des festen Holdout-Splits mit begrenzter szenariospezifischer Testabdeckung zu interpretieren.

Analyse der False-Negative-Cases. Die drei nicht detektierten positiven Cases (FN = 3) stammen aus mali/scenario_6 (*Rootkit / Backdoor*). Bei dem auf Validation ausgewählten Threshold liegen diese Cases unterhalb der Entscheidungsgrenze. Eine niedrigere Schwelle könnte den Test-Recall grundsätzlich erhöhen, würde jedoch voraussichtlich zusätzliche False Positives erzeugen. Dieser Trade-off

wird in der Threshold-Sensitivity sichtbar. Die ergänzende Szenarioanalyse zeigt zudem, dass `mali/scenario_6` mit 21 von 24 korrekt erkannten Cases (Recall 0,8750) insgesamt schwieriger ist, jedoch nicht vollständig verfehlt wird.

Evaluation auf realem Produktions-Traffic. CloudAnoBench verwendet kontrollierte Anomalie-Injektionen. Ob die gelernten Muster auf reale Infrastrukturdaten mit natürlichen Klassenverteilungen, variablen Log-Formaten und unbekannten Anomalietypen übertragbar sind, bleibt eine wichtige offene Frage.

Die Arbeit zeigt damit weniger eine universelle Verbesserung durch Fusion als vielmehr die Notwendigkeit, Log-Kontext und Evaluationsgranularität in der Cloud-Anomaly-Detection sorgfältig zu berücksichtigen und dabei methodisch sauber zwischen dem Beitrag einzelner Komponenten und dem Beitrag ihrer Kombination zu unterscheiden.

Literatur

- [1] Bushra A. Alahmadi, Louise Axon und Ivan Martinovic. „99% False Positives: A Qualitative Study of SOC Analysts’ Perspectives on Security Alarms“. In: *31st USENIX Security Symposium (USENIX Security 22)*. 2022, S. 2783–2800.
- [2] Daniel Arp u. a. „Dos and Don’ts of Machine Learning in Computer Security“. In: *31st USENIX Security Symposium (USENIX Security 22)*. 2022, S. 3971–3988.
- [3] Gavin C. Cawley und Nicola L. C. Talbot. „On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation“. In: *Journal of Machine Learning Research* 11 (2010), S. 2079–2107.
- [4] Tianqi Chen und Carlos Guestrin. „XGBoost: A Scalable Tree Boosting System“. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, S. 785–794. DOI: 10.1145/2939672.2939785.
- [5] Jesse Davis und Mark Goadrich. „The Relationship Between Precision-Recall and ROC Curves“. In: *Proceedings of the 23rd International Conference on Machine Learning (ICML)*. ACM, 2006, S. 233–240. DOI: 10.1145/1143844.1143874.
- [6] Haibo He und Eduardo A. Garcia. „Learning from Imbalanced Data“. In: *IEEE Transactions on Knowledge and Data Engineering* 21.9 (2009), S. 1263–1284. DOI: 10.1109/TKDE.2008.239.
- [7] Shilin He u. a. „A Survey on Automated Log Analysis for Reliability Engineering“. In: *ACM Computing Surveys* 54.6 (2022), 130:1–130:37. DOI: 10.1145/3460345.
- [8] Peter J. Huber. *Robust Statistics*. 1. Aufl. John Wiley & Sons, 1981.
- [9] Shachar Kaufman u. a. „Leakage in Data Mining: Formulation, Detection, and Avoidance“. In: *ACM Transactions on Knowledge Discovery from Data* 6.4 (2012), 15:1–15:21. DOI: 10.1145/2382577.2382579.
- [10] Josef Kittler u. a. „On Combining Classifiers“. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 20.3 (1998), S. 226–239. DOI: 10.1109/34.667881.
- [11] F. Pedregosa u. a. „Scikit-learn: Machine Learning in Python“. In: *Journal of Machine Learning Research* 12 (2011), S. 2825–2830.

- [12] David R. Roberts u. a. „Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure“. In: *Ecography* 40.8 (2017), S. 913–929. DOI: 10.1111/ecog.02881.
- [13] Takaya Saito und Marc Rehmsmeier. „The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets“. In: *PLOS ONE* 10.3 (2015), e0118432. DOI: 10.1371/journal.pone.0118432.
- [14] Shahroz Tariq u. a. „Alert Fatigue in Security Operations Centres: Research Challenges and Opportunities“. In: *ACM Computing Surveys* 57.9 (Apr. 2025), S. 1–38. DOI: 10.1145/3723158.
- [15] David M. J. Tax u. a. „Combining multiple classifiers by averaging or by multiplying?“ In: *Pattern Recognition* 33.9 (2000), S. 1475–1485. DOI: 10.1016/S0031-3203(99)00138-7.
- [16] Xinkai Zou u. a. *Towards Generalizable Context-aware Anomaly Detection: A Large-scale Benchmark in Cloud Environments*. arXiv:2508.01844. 2025.

A Anhang

Die zur Reproduktion relevanten Artefakte befinden sich im Repository: die fünf Jupyter-Notebooks (in dieser Reihenfolge auszuführen: 01_data_exploration, 02_metric_baseline, 03_log_component, 04_fusion, 05_error_analysis), die erzeugten Abbildungen und Artefakt-CSVs im Verzeichnis docs/ sowie die 67 automatisierten Tests in 14 Testklassen in tests/test_pipeline.py. Notebook 05 umfasst die Fehleranalyse: False Negatives, False Positives nach Szenario, malicious-Test-Szenario-Recall und Reproduzierbarkeitschecks.

Ergänzend liegt im Repository ein Dockerfile vor, das eine einfache Jupyter-Lab-Umgebung auf Basis von Python 3.12 mit den benötigten Abhängigkeiten bereitstellt. Es dient der lokalen Reproduzierbarkeit der Notebook-Ausführung und ist nicht als produktionsreife Deployment-Umgebung zu verstehen. Der Container kann aus dem Repository-Wurzelverzeichnis mit

```
docker build -t cloudanobench-thesis .
```

gebaut und anschließend mit

```
docker run -p 8888:8888 cloudanobench-thesis
```

gestartet werden.

Quellcode und Reproduzierbarkeit. Der Quellcode, die ausgeführten Notebooks, die automatisierten Tests und die kanonischen Ergebnisartefakte sind öffentlich unter <https://github.com/Nirosh04/bachelorarbeit-cloudanobench> verfügbar. Der für die in dieser Arbeit berichteten Ergebnisse verwendete reproduzierbare Code- und Ergebnisstand entspricht dem vollständigen Git-Commit b416cb1d54d46adbe2f72d75d0eae4c752e87adc. Zusätzlich ist dieser Stand als Git-Tag thesis-reproducible-v7 referenziert.

A.1 Dokumentation der KI-Nutzung

Verwendete Werkzeuge und Einsatzbereiche

Im Verlauf der Arbeit wurden drei generative KI-Werkzeuge unterstützend eingesetzt. Die folgende Tabelle gibt einen Überblick.

Werkzeug	Anbieter	Modell	Nutzungszeitraum
ChatGPT	OpenAI	GPT-5.5	05/2026–07/2026
Claude	Anthropic	Claude Sonnet 4.6	05/2026–07/2026
Perplexity	Perplexity AI	nicht durchgängig protokolliert	05/2026

ChatGPT (Anbieter: OpenAI, Modell: GPT-5.5) wurde eingesetzt, um selbst verfasste Textpassagen auf Verständlichkeit und sprachliche Präzision zu prüfen, bereits entwickelte methodische Überlegungen kritisch zu hinterfragen und einzelne Angaben auf Konsistenz zu kontrollieren.

Claude (Anbieter: Anthropic, Modell: Claude Sonnet 4.6) wurde über die Entwicklungsumgebung Cursor zur Prüfung und punktuellen Verbesserung bestehenden Codes, zur Analyse technischer Datenflüsse sowie zur Unterstützung bei Tests und repositoryweiten Konsistenzkontrollen eingesetzt.

Perplexity (Anbieter: Perplexity AI) wurde ausschließlich als ergänzendes Werkzeug bei der Recherche potenziell relevanter Literatur eingesetzt. Auswahl, Lektüre, Bewertung und Zitierung der tatsächlich verwendeten Quellen wurden anhand der Originalquellen vorgenommen.

Repräsentative Beispiele

Die folgenden drei Beispiele sind repräsentative, gekürzte Ausschnitte aus tatsächlichen Interaktionen. Sie decken nicht sämtliche KI-Nutzungen im Verlauf der Arbeit ab.

Beispiel 1: Prüfung der Imputationslogik

Auszug aus dem Prompt:

Prüfe ausschließlich lesend, ob fehlende Metrikwerte erst nach Anwendung des festen Train-Validation-Test-Splits imputiert werden und ob die dafür verwendeten Mediane ausschließlich aus Trainingsdaten stammen. Verändere keine Dateien. Benenne mögliche Leakage-Pfade und die betroffenen Verarbeitungsschritte.

Auszug aus der Antwort:

Die fehlenden Werte werden bereits in der Vorverarbeitung vor Anwendung des Splits ersetzt. Dadurch fließen auch Werte aus Validation und Test in die Medianberechnung ein. Die später vorgesehene Train-only-Imputation kann deshalb auf diese fehlenden Werte nicht mehr wirken.

Eigene Prüfung:

Die betreffenden Verarbeitungsschritte wurden anschließend direkt in den Notebooks überprüft. Dabei bestätigte sich, dass die fehlenden Werte bereits vor Anwendung der Split-Zuordnungen ersetzt worden waren. Anschließend wurde der Ablauf korrigiert und um sechs Regressionstests ergänzt, die sicherstellen, dass die Imputationsparameter ausschließlich aus Trainingsdaten bestimmt werden.

Auswirkung auf die Endfassung:

Die Reihenfolge der Vorverarbeitung wurde korrigiert und alle davon abhängigen Ergebnisartefakte wurden erneut erzeugt. Einzelne Kennzahlen der Metrik-Baseline und der Soft-Fusion änderten sich geringfügig. Die zentrale Schlussfolgerung der Arbeit blieb unverändert.

Beispiel 2: Typografische Prüfung einer Abbildungsunterschrift

Auszug aus dem Prompt:

Führe ausschließlich einen gezielten typografischen und technischen Konsistenzcheck durch. Prüfe insbesondere die mathematische Schreibweise von FPR_{anom} und FPR_{norm} in der Bildunterschrift von Abbildung 7. Verändere keine Ergebnisse, Zahlen oder methodischen Aussagen.

Auszug aus der Antwort:

In der Bildunterschrift wurde eine inkonsistente Indexschreibweise gefunden. Die Bezeichnungen wurden durch die im übrigen Dokument verwendete mathematische Notation ersetzt. Datenwerte und Aussagen blieben unverändert.

Eigene Prüfung:

Die korrigierte Notation wurde mit den übrigen Formeln und Tabellen der Bachelorarbeit abgeglichen. Anschließend wurde die PDF erneut mit $\text{\LaTeX}$ und Biber erstellt und Referenzen, Zitate und Seitenlayout wurden geprüft.

Auswirkung auf die Endfassung:

Ausschließlich die mathematische Darstellung der beiden FPR-Indizes in der Bildunterschrift wurde vereinheitlicht. Abbildung, Datenwerte und Interpretation blieben unverändert.

Beispiel 3: Abgleich einer Caption mit Ergebnisartefakten

Auszug aus dem Prompt:

Führe ausschließlich einen Read-only-Konsistenzcheck durch. Prüfe, ob die Ergebniszahlen in der Bachelorarbeit mit den kanonischen CSV-Artefakten übereinstimmen. Ändere keine Datei und nenne bei einer Abweichung die genaue Fundstelle.

Auszug aus der Antwort:

In der Bildunterschrift der Precision-Recall-Kurve der Metrik-Baseline steht noch $AP = 0,2439$. Der aktuelle Wert gemäß den Ergebnisartefakten und der Ergebnistabelle beträgt $AP = 0,2652$.

Eigene Prüfung:

Der genannte Wert wurde mit `docs/final_results_table.csv`, `docs/metric_case_results.csv` und der zugehörigen Ergebnistabelle der Bachelorarbeit abgeglichen. Alle drei Quellen bestätigten den Wert 0,2652. Anschließend wurde ausschließlich die veraltete Bildunterschrift korrigiert.

Auswirkung auf die Endfassung:

Die AP-Angabe in der Bildunterschrift wurde von 0,2439 auf 0,2652 aktualisiert. Die Berechnung, die Ergebnisartefakte und alle übrigen Zahlen blieben unverändert.

Eigenleistung und Verantwortlichkeit

Die Festlegung des finalen Evaluationsdesigns, die Durchführung der Experimente, die Prüfung der erzeugten Ergebnisse sowie deren Interpretation wurden im Rahmen dieser Arbeit eigenständig vom Verfasser dieser Arbeit verantwortet. Generative KI wurde als unterstützendes Werkzeug für Formulierungsalternativen, kritische Gegenprüfungen, Code- und Konsistenzprüfungen sowie die Recherche potenziell relevanter Literatur eingesetzt. KI-Vorschläge wurden kritisch anhand des Quellcodes, der Tests, der erzeugten Ergebnisartefakte, der Originalquellen und des Betreuerfeedbacks geprüft und nicht ungeprüft übernommen. Die Verantwortung für sämtliche Aussagen, Berechnungen und Schlussfolgerungen liegt beim Verfasser dieser Arbeit.

Reflexion

Der Einsatz generativer KI-Werkzeuge hatte in dieser Arbeit sowohl erkennbaren Nutzen als auch klar erfahrbare Grenzen.

Als nützlich erwies sich vor allem die Geschwindigkeit bei technischen Such- und Konsistenzprüfungen. Repositoryweite Diff-Audits, die manuell sehr aufwendig gewesen wären, konnten strukturierter angegangen werden. Ebenso nützlich war die Möglichkeit, alternative Formulierungen für einen bereits selbst erarbeiteten Sachverhalt

einzuholen und dann eigenständig zu entscheiden, welche Variante wissenschaftlich präziser und angemessener ist. Die Unterstützung bei der systematischen Prüfung von Code und Artefakten erleichterte es, potenzielle Inkonsistenzen überhaupt erst als prüfenswert zu identifizieren.

Gleichzeitig wurden die Grenzen solcher Werkzeuge konkret erfahrbar. KI-Vorschläge können fachlich falsch, zu weitgehend oder für den tatsächlichen Arbeitskontext unangemessen sein. Frühere Vorschläge enthielten etwa Hinweise auf zusätzliche Experimente oder Anforderungen, die sich an einer publikationsorientierten Forschungsarbeit orientierten und für den Rahmen einer Bachelorarbeit weder notwendig noch sinnvoll waren. Auch Ergebnisvergleiche wurden gelegentlich ungenau formuliert: Ein Vergleich zweier Systemvarianten wurde beispielsweise verfehlt, indem „zwei weniger False Positives“ und „zwei zusätzliche False Positives“ verwechselt wurden. Solche Fehler zeigen, dass KI-generierte Aussagen über quantitative Ergebnisse zwingend gegen die tatsächlichen Artefakte geprüft werden müssen.

Um diesen Risiken zu begegnen, wurden folgende Kontrollmechanismen konsequent eingesetzt: Für Literatur wurden die Originalquellen gelesen und bewertet, ohne KI-Zusammenfassungen zu übernehmen. Quantitative Angaben wurden stets gegen die kanonischen CSV-Artefakte geprüft, nicht gegen gerundete oder verbal zusammengefasste Berichte. Für Code- und Methodenprüfungen dienten Git-Diffs, die 67 automatisierten Tests, die Notebook-Validierung sowie der $\text{\LaTeX}$ /Biber-Build als unabhängige Prüfinstanzen. Das Betreuerfeedback diente als wichtige externe Korrektur, insbesondere wenn Formulierungen zu stark oder methodisch nicht hinreichend präzise waren.

Die wichtigste Lernerfahrung ist, dass KI-Ausgaben als prüfbare Vorschläge zu behandeln sind, nicht als wissenschaftliche Autorität. Die Nutzung solcher Werkzeuge hat die Bedeutung eigener fachlicher Kontrolle nicht vermindert, sondern in gewisser Weise erhöht: Da Vorschläge plausibel klingen, aber dennoch falsch sein können, erfordert jeder übernommene Vorschlag eine explizite Entscheidung und eine nachvollziehbare Begründung. Die Verantwortung für den finalen Stand der Arbeit, für ihre Aussagen, Berechnungen, Methodik und Schlussfolgerungen, lag zu jedem Zeitpunkt beim Verfasser dieser Arbeit.